Relatório extensionista

Disciplina: Programação para dispositivos móveis em android

Aluno: Integrante do grupo (dados omitidos)

Matrícula: omitida na versão pública

Aluno: Integrante do grupo (dados omitidos)

Matrícula: omitida na versão pública

Aluno: Integrante do grupo (dados omitidos)

Matrícula: omitida na versão pública

Aluno: Integrante do grupo (dados omitidos)

Matrícula: omitida na versão pública

Curso: Análise e Desenvolvimento de Sistemas

Período de Atividade: 23/02/2026 a 08/06/2026

Relatório Extensionista

Curso	Análise e Desenvolvimento de Sistemas		
Aluno	Integrante do grupo (dados omitidos)	Matrícula	Omitida
Aluno	Integrante do grupo (dados omitidos)	Matrícula	Omitida
Aluno	Integrante do grupo (dados omitidos)	Matrícula	Omitida
Aluno	Integrante do grupo (dados omitidos)	Matrícula	Omitida
Orientador	Paulo Cruz		
Período da Atividade	23/02/2026 a *08/06/2026*		

Sumário

1. Introdução 2,3
2. Contexto da Empresa 3
3. Objetivos a serem alcançados pela atividade extensionista 3
4. Atividades Desenvolvidas 4,5
5. Principais Aprendizados 6
6. Resultados Obtidos 6
7. Conclusão 7
8. Agradecimentos 7
9. Aceite da Empresa 7,8
10. Anexos 9,10,11,12

1. Introdução

Neste relatório, apresentam-se as atividades desenvolvidas durante o projeto extensionista no campo de Análise e Desenvolvimento de Sistemas. Esta atividade foi realizada como requisito parcial para a conclusão da disciplina de Programação Para Dispositivos Móveis em Android, integrante da matriz curricular da graduação em Análise e Desenvolvimento de Sistemas, ofertada pela Universidade Estácio – Unidade Liberdade.

No âmbito deste trabalho, desenvolveu-se um projeto de software completo cujo principal escopo consistiu na criação de um ecossistema digital integrado — composto por uma aplicação web e um aplicativo móvel —, direcionado a uma clínica de atendimento especializado em saúde mental.

A proposta buscou modernizar e otimizar os processos operacionais internos da instituição, além de aprimorar significativamente a experiência dos pacientes no que tange ao acesso e à usabilidade dos serviços oferecidos.

O sistema concebido tem como foco central a automatização e a facilitação do agendamento de consultas, viabilizando que os pacientes realizem marcações, remarcações ou cancelamentos de atendimentos de maneira intuitiva, célere e segura, mitigando a necessidade de suporte presencial ou telefônico.

Ademais, o projeto contempla funcionalidades estratégicas para auxiliar o corpo profissional e técnico da clínica no gerenciamento de fluxos de atendimento, prontuários digitais e históricos clínicos dos pacientes. Para garantir a integridade, a organização e a eficiência no manejo das informações, estruturou-se um banco de dados relacional robusto. Este repositório centraliza o armazenamento de dados críticos ao funcionamento do estabelecimento, tais como o cadastro de usuários, o registro de profissionais de saúde, a grade de horários disponíveis e o histórico de consultas.

Tal centralização mitiga falhas de comunicação, reduz a perda de informações e elimina inconsistências documentais. Outro pilar fundamental do projeto reside no compromisso com a segurança e a confidencialidade das informações tratadas. Por se tratar de dados sensíveis associados à saúde mental dos indivíduos, foram rigorosamente consideradas as boas práticas de proteção de dados, mecanismos de controle de acesso autenticado e diretrizes de privacidade de software. No que tange ao ecossistema tecnológico empregado, a arquitetura de backend e a integração com o banco de dados foram estruturadas utilizando a linguagem Python, assegurando robustez, escalabilidade e confiabilidade ao microsserviço. Na camada

de frontend e interface de usuário, aplicou-se a linguagem JavaScript, proporcionando uma experiência interativa, dinâmica e responsiva tanto na plataforma web quanto na aplicação mobile.

Portanto, este projeto não se limitou à mera entrega de uma solução tecnológica, mas propôs uma efetiva transformação digital para a clínica parceira, promovendo maior eficiência operacional, estreitando a comunicação entre a equipe e seus assistidos, e elevando o padrão de qualidade do atendimento prestado.

2. Contexto da Empresa

Nome da Empresa: Estácio

CNPJ da Empresa: omitido

Nome do representante da empresa: omitido na versão pública

Localização: omitido

Setor de Atuação: Psiquiatria

3. Objetivos a serem alcançados pela atividade extensionista

Desenvolver uma plataforma web e um aplicativo móvel integrado voltados para uma clínica de saúde mental, com o propósito de elevar a confiabilidade e a transparência dos serviços ofertados aos pacientes. Para tanto, estabeleceu-se como meta a implementação de um sistema versátil de agendamentos, um módulo de autenticação baseado em documentação oficial e uma interface focada em usabilidade e acessibilidade para o usuário final.

4. Atividades Desenvolvidas

Durante o período de vigência da atividade extensionista, executaram-se as seguintes etapas de engenharia de software:

Planejamento: Elaboração da arquitetura de design de interface (UI/UX) do site e do aplicativo, bem como o mapeamento e a definição do ecossistema tecnológico a ser empregado.

Análise de Requisitos:

R1: Automação de processos para mitigar gargalos operacionais decorrentes do elevado fluxo de clientes, exigindo a modelagem de um banco de dados robusto.

R2: Módulo de cadastro de usuários e controle de acesso ao sistema de agendas; desenvolvimento de interface mobile dedicada aos estagiários e funcionalidade de notificações para manter os pacientes cientes de cronogramas e consultas. O sistema deve gerenciar o encaminhamento triado do paciente e alinhar as agendas acadêmicas, definindo os dias específicos de atendimento dos estagiários e evitando divergências informacionais.

R2.1: Implementação de subsistema de avaliação de desempenho voltado aos estagiários e de métricas de satisfação da clínica.

R2.2: Desenvolvimento de gatilho automático via API do WhatsApp para envio de notificações semestrais aos pacientes, consultando o interesse na continuidade do tratamento na instituição.

R3: Regra de negócio limitando o atendimento inicial do estagiário a apenas 1 (um) paciente, progredindo de forma escalonada até o limite de 3 (três) pacientes simultâneos conforme evolução de nível.

Definição de rigidez de horários: caso o atendimento inicial ocorra em uma segunda-feira às 18h, o horário permanecerá fixo durante o ciclo do tratamento (composto, por exemplo, por 10 consultas).

Gerenciamento físico de espaço: infraestrutura contendo 8 salas, sendo a Sala 2 restrita por horário, a Sala 1 dedicada exclusivamente à área de nutrição, e 6 salas de livre disponibilidade para consultas gerais.

Dimensionamento de pessoal: 94 estagiários e 6 supervisores. Horário de funcionamento: Segunda a sexta-feira, das 08:00 às 20:00.

Cronograma: 2 meses

Semana 1 – Planejamento e Ideação

Semana 2 – Configuração do Ambiente e Estrutura Inicial

Semana 3 – Cadastro de Pacientes

Semana 4 – Tela de vitrine e Edição dos Pacientes

Semana 5 – Registro de Consultas

Semana 6 – Relatórios e Intermediários

Semana 7 – Testes e Ajustes

Semana 8 – Apresentação Final

Desenvolvimento de Software: Codificação e implementação de módulos específicos do sistema através das linguagens JavaScript, Python, do framework React Native e do sistema gerenciador de banco de dados MySQL. Como Ambiente de Desenvolvimento Integrado (IDE), utilizou-se o Visual Studio Code.

Testes de Software: Execução de baterias de testes focadas em segurança da informação, testes de integração de módulos e verificação de responsividade de interface. Realizaram-se, complementarmente, testes de carga e consistência no banco de dados mediante a inserção de dados fictícios (mock data).

Suporte Técnico: Simulação de suporte técnico aos usuários finais do ecossistema, voltada ao diagnóstico, triagem e resolução de falhas e bugs reportados.

5. Principais Aprendizados

No decorrer da execução do projeto extensionista, destacaram-se os seguintes aspectos de desenvolvimento profissional e acadêmico:

Otimização das soft skills, com ênfase no trabalho em equipe, colaboração e comunicação interpessoal.

Aprofundamento técnico e domínio prático na utilização do framework MySQL e na codificação em Python.

Amadurecimento na compreensão teórica e prática sobre os processos de documentação técnica de software e engenharia de requisitos.

6. Resultados Obtidos

1. O projeto iniciou-se com o mapeamento e o levantamento detalhado de requisitos, definindo-se as linguagens de programação, ferramentas de IDE e padrões de design corporativo. O processo estimulou a criatividade no desenvolvimento de interfaces UI/UX e aprimorou o raciocínio lógico de programação do grupo. Como ponto de complexidade, destacou-se a curva de aprendizado necessária para a assimilação e o domínio do framework React Native.

2. Obteve-se êxito no desenvolvimento de uma aplicação móvel e de uma plataforma web totalmente funcionais, caracterizadas por uma interface limpa e intuitiva, que cumprem com eficiência o propósito de otimizar e automatizar o fluxo de agendamento de consultas da instituição.

3. Papéis e responsabilidade:

Front-End: integrante do grupo

Back-End/Banco de dados: integrantes do grupo

Documentação: integrante responsável pela documentação

7. Conclusão

A execução deste projeto evidenciou a centralidade do trabalho em equipe e da comunicação assertiva entre os integrantes como pilares fundamentais para a entrega de um produto de software organizado, padronizado e adequadamente documentado.

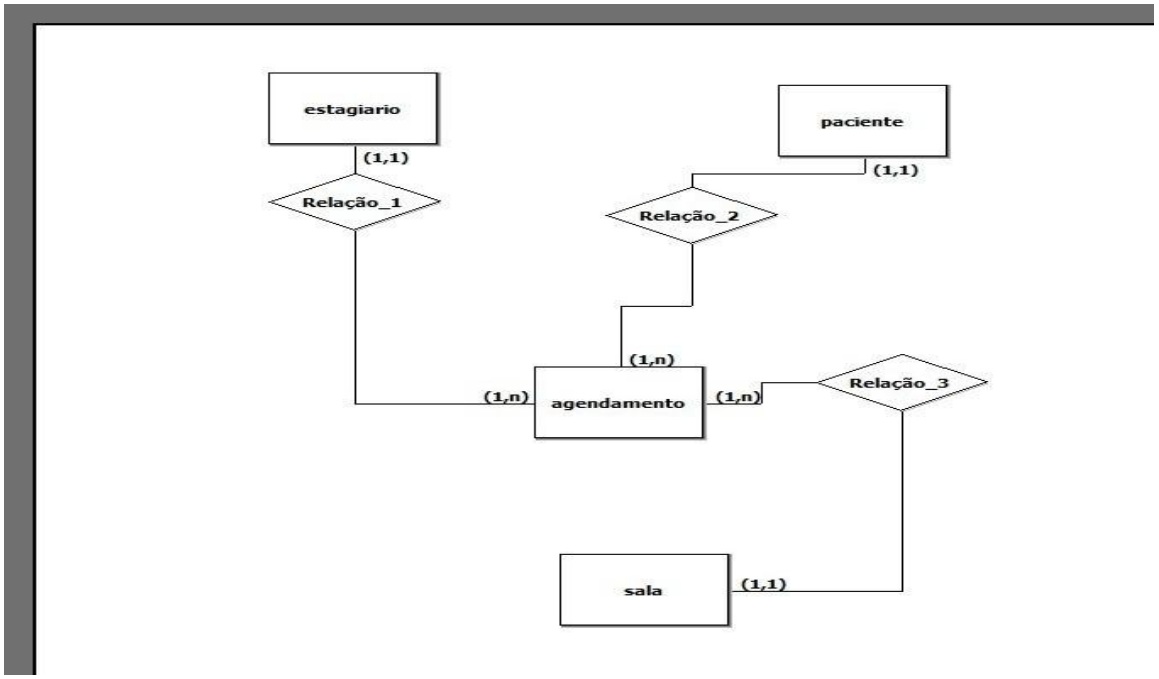
A atividade extensionista proporcionou o ambiente ideal para a aplicação prática dos conhecimentos teóricos adquiridos em sala de aula, consolidando competências técnicas em engenharia de software. Com a conclusão do sistema, os profissionais da saúde e a equipe de gestão clínica dispõem agora de ferramentas integradas (web e mobile) capazes de acelerar, simplificar e conferir precisão ao processo de agendamentos.

Esses resultados foram alcançados através do aproveitamento máximo das ferramentas e tecnologias mapeadas na fase de requisitos, convergindo inovação tecnológica, fundamentação teórica e sinergia de equipe em prol de uma solução de impacto social e institucional.

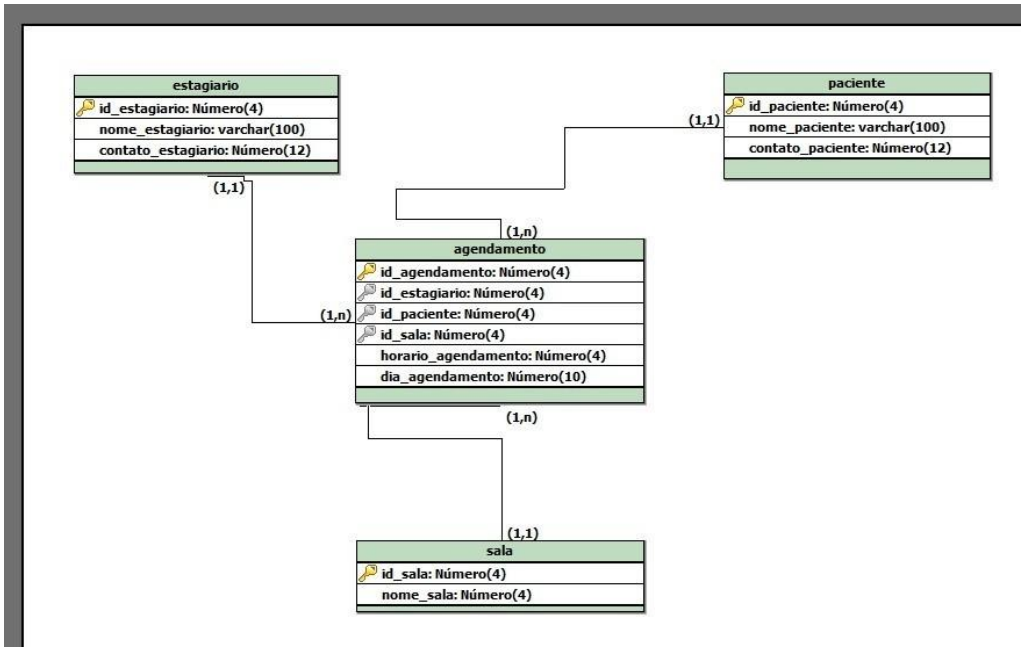
8. Agradecimentos

Gostariamos de expressar nossa gratidão à equipe da Clinica e Paulo da Cruz pela oportunidade concedida e pelo apoio prestado durante o período de realização do projeto

Modelo Conceitual:



Modelo Logico:



Banco de dados/MySql:

bd_clinica

Limit to 1000 rows

```sql
-- ========================================================================
-- SCRIPT DE CONFIGURAÇÃO E MODELAGEM DO BANCO DE DADOS: clinica_db
-- ========================================================================

-- Cria a base de dados se ela ainda não existir no sistema
CREATE DATABASE IF NOT EXISTS clinica_db;
USE clinica_db;

-- ------------------------------------------------------------------------
-- 1. TABELA: ESTAGIARIOS
-- Motivo da Ordem: Deve ser criada primeiro porque não possui dependências.
-- ------------------------------------------------------------------------
CREATE TABLE `estagiarios` (
  `id` int(11) NOT NULL AUTO_INCREMENT,              -- Chave primária auto-incrementável (ID único do estagiário)
  `nome` varchar(100) NOT NULL,                      -- Nome do estagiário (campo obrigatório)
  `data_nascimento` date NOT NULL DEFAULT '2000-01-01', -- Valor fallback para evitar quebras se o front não enviar
  `email` varchar(100) NOT NULL,                     -- E-mail para login
```

bd_clinica

Limit to 1000 rows

```sql
  `data_nascimento` date NOT NULL DEFAULT '2000-01-01', -- Valor fallback para evitar quebras se o front não enviar
  `email` varchar(100) NOT NULL,                    -- E-mail para login
  `senha` varchar(255) NOT NULL,                    -- Senha em texto puro (ou hash criptográfico)
  `cargo` varchar(50) DEFAULT 'Estagiário de Clínica', -- Atribuição padrão se omitida
  PRIMARY KEY (`id`),                               -- Define o ID como identificador principal
  UNIQUE KEY `email` (`email`)                      -- Restrição UNIQUE: Impede o cadastro de dois estagiários com o mesmo e-mail
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;


-- ------------------------------------------------------------------------
-- 2. TABELA: PACIENTES
-- Motivo da Ordem: Também é independente. Guarda os prontuários e dados clínicos.
-- ------------------------------------------------------------------------
CREATE TABLE `pacientes` (
  `id` int(11) NOT NULL AUTO_INCREMENT,              -- ID único do paciente
  `nome_completo` varchar(150) NOT NULL,             -- Nome civil completo
  `data_nascimento` date NOT NULL DEFAULT '2000-01-01', -- Evita erros de gravação no formulário mobile
  `cpf` varchar(14) NOT NULL,                        -- CPF com ou sem máscara (Ex: 000.000.000-00)
  `sexo` varchar(20) DEFAULT NULL,                   -- Gênero ou identificação biológica
  `email` varchar(100) DEFAULT NULL,                 -- E-mail do paciente (opcional)
  `telefone` varchar(15) DEFAULT NULL,               -- Contato telefônico para avisos de consulta
  `convenio` varchar(50) DEFAULT NULL,               -- Plano de saúde ou "Particular"
  `situacao` varchar(30) DEFAULT NULL,               -- Status clínico (Ex: Em triagem, Crônico, Alta)
  `observacoes` text DEFAULT NULL,                   -- Histórico médico ou restrições do paciente
```

bd_clinica

Limit to 1000 rows

```
36    `telefone` varchar(15) DEFAULT NULL,                -- Contato telefônico para avisos de consulta
37    `convenio` varchar(50) DEFAULT NULL,                -- Plano de saúde ou "Particular"
38    `situacao` varchar(30) DEFAULT NULL,                -- Status clínico (Ex: Em triagem, Crônico, Alta)
39    `observacoes` text DEFAULT NULL,                    -- Histórico médico ou restrições do paciente
40    PRIMARY KEY (`id`),
41    UNIQUE KEY `cpf` (`cpf`)                            -- Restrição UNIQUE: Bloqueia CPFs duplicados no sistema
42  ) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
43
44
45  -- ------------------------------------------------------------------------
46  -- 3. TABELA: SALAS
47  -- Motivo da Ordem: Registra os locais físicos de atendimento antes do agendamento.
48  -- ------------------------------------------------------------------------
49  CREATE TABLE `salas` (
50    `id` int(11) NOT NULL AUTO_INCREMENT,               -- ID único da sala
51    `nome_sala` varchar(50) NOT NULL,                   -- Nome visível (Ex: Consultório A, Ala Sul)
52    PRIMARY KEY (`id`),
53    UNIQUE KEY `nome_sala` (`nome_sala`)                -- Impede a criação de salas com nomes idênticos
54  ) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
55
56
57  -- ------------------------------------------------------------------------
58  -- 4. TABELA: AGENDAMENTOS (A tabela central de relacionamento)
59  -- Motivo da Ordem: Deve ser criada por último, pois depende dos IDs das 3 tabelas anteriores.
```

bd_clinica

Limit to 1000 rows

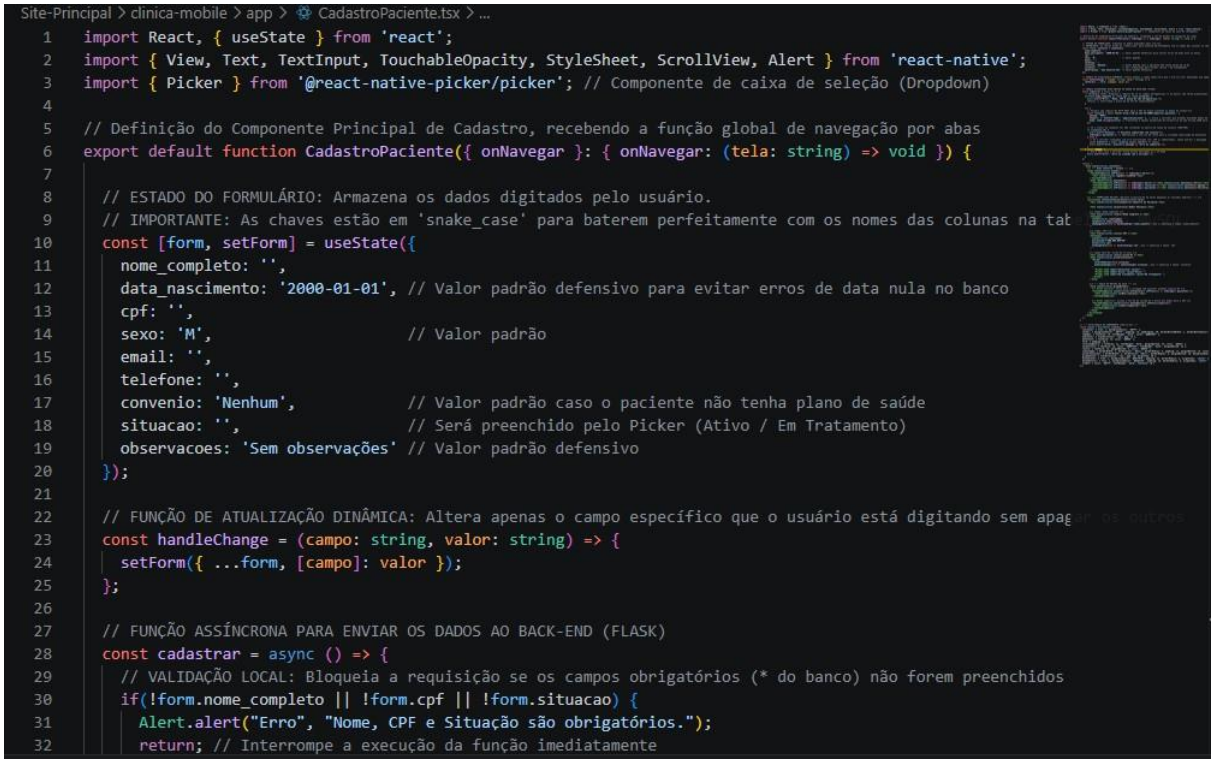
```
61  CREATE TABLE `agendamentos` (
62    `id` int(11) NOT NULL AUTO_INCREMENT,               -- ID único do agendamento de consulta
63    `paciente_id` int(11) NOT NULL,                     -- FK: Associa a consulta a um Paciente
64    `estagiario_id` int(11) NOT NULL,                   -- FK: Associa quem marcou/atenderá a consulta
65    `sala_id` int(11) NOT NULL,                         -- FK: Associa o local físico do atendimento
66    `data_consulta` date NOT NULL,                       -- Data (AAAA-MM-DD)
67    `horario` time NOT NULL,                            -- Horário (HH:MM:SS)
68    `tipo_consulta` varchar(50) DEFAULT NULL,           -- Modalidade (Ex: Retorno, Primeira Consulta, Exame)
69    PRIMARY KEY (`id`),
70
71    -- Criação de Índices (Keys) para otimizar a velocidade de busca em consultas com INNER JOIN
72    KEY `paciente_id` (`paciente_id`),
73    KEY `estagiario_id` (`estagiario_id`),
74    KEY `sala_id` (`sala_id`),
75
76    -- REGRAS DE INTEGRIDADE REFERENCIAL (Chaves Estrangeiras)
77    -- ON DELETE CASCADE: Se o paciente ou o estagiário for deletado do sistema, o MySQL apaga todos os
78    -- agendamentos vinculados a eles de forma automática, impedindo registros órfãos no banco.
79    CONSTRAINT `agendamentos_ibfk_1` FOREIGN KEY (`paciente_id`) REFERENCES `pacientes` (`id`) ON DELETE CASCADE,
80    CONSTRAINT `agendamentos_ibfk_2` FOREIGN KEY (`estagiario_id`) REFERENCES `estagiarios` (`id`) ON DELETE CASCADE,
81
82    -- Para as salas, não usamos o CASCADE por segurança, impedindo que apaguem uma sala cheia de consultas sem querer.
83    CONSTRAINT `agendamentos_ibfk_3` FOREIGN KEY (`sala_id`) REFERENCES `salas` (`id`)
84  ) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
  -- Para as salas, não usamos o CASCADE por segurança, impedindo que apaguem uma sala cheia de consultas sem querer.
  CONSTRAINT `agendamentos_ibfk_3` FOREIGN KEY (`sala_id`) REFERENCES `salas` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;


-- --------------------------------------------------------------------------
-- 5. POPULAÇÃO INICIAL DE DADOS (Carga Inicial)
-- Popula a tabela de salas para que os seletores (Pickers) do aplicativo
-- já tenham opções disponíveis no primeiro uso.
-- --------------------------------------------------------------------------
INSERT INTO `salas` (`nome_sala`) VALUES
('Consultório 1'),
('Consultório 2'),
('Triagem');
```

Código/React-Native

Cadastrar Paciente:



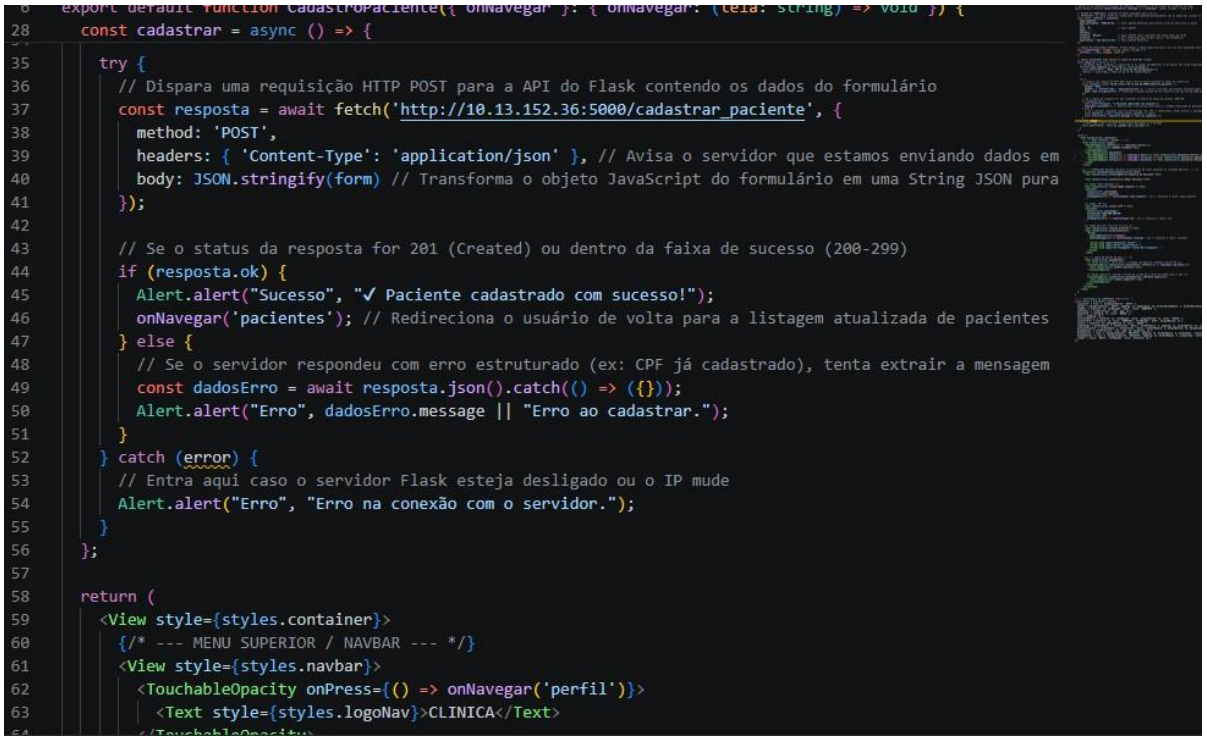
```
Site-Principal > clinica-mobile > app > CadastroPaciente.tsx > ...
import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, ScrollView, Alert } from 'react-native';
import { Picker } from '@react-native-picker/picker'; // Componente de caixa de seleção (Dropdown)

// Definição do Componente Principal de Cadastro, recebendo a função global de navegação por abas
export default function CadastroPaciente({ onNavegar }: { onNavegar: (tela: string) => void }) {

  // ESTADO DO FORMULÁRIO: Armazena os dados digitados pelo usuário.
  // IMPORTANTE: As chaves estão em 'snake_case' para baterem perfeitamente com os nomes das colunas na tab
  const [form, setForm] = useState({
    nome_completo: '',
    data_nascimento: '2000-01-01', // Valor padrão defensivo para evitar erros de data nula no banco
    cpf: '',
    sexo: 'M',                     // Valor padrão
    email: '',
    telefone: '',
    convenio: 'Nenhum',            // Valor padrão caso o paciente não tenha plano de saúde
    situacao: '',                  // Será preenchido pelo Picker (Ativo / Em Tratamento)
    observacoes: 'Sem observações' // Valor padrão defensivo
  });

  // FUNÇÃO DE ATUALIZAÇÃO DINÂMICA: Altera apenas o campo específico que o usuário está digitando sem apag
  const handleChange = (campo: string, valor: string) => {
    setForm({ ...form, [campo]: valor });
  };

  // FUNÇÃO ASSÍNCRONA PARA ENVIAR OS DADOS AO BACK-END (FLASK)
  const cadastrar = async () => {
    // VALIDAÇÃO LOCAL: Bloqueia a requisição se os campos obrigatórios (* do banco) não forem preenchidos
    if(!form.nome_completo || !form.cpf || !form.situacao) {
      Alert.alert("Erro", "Nome, CPF e Situação são obrigatórios.");
      return; // Interrompe a execução da função imediatamente
```

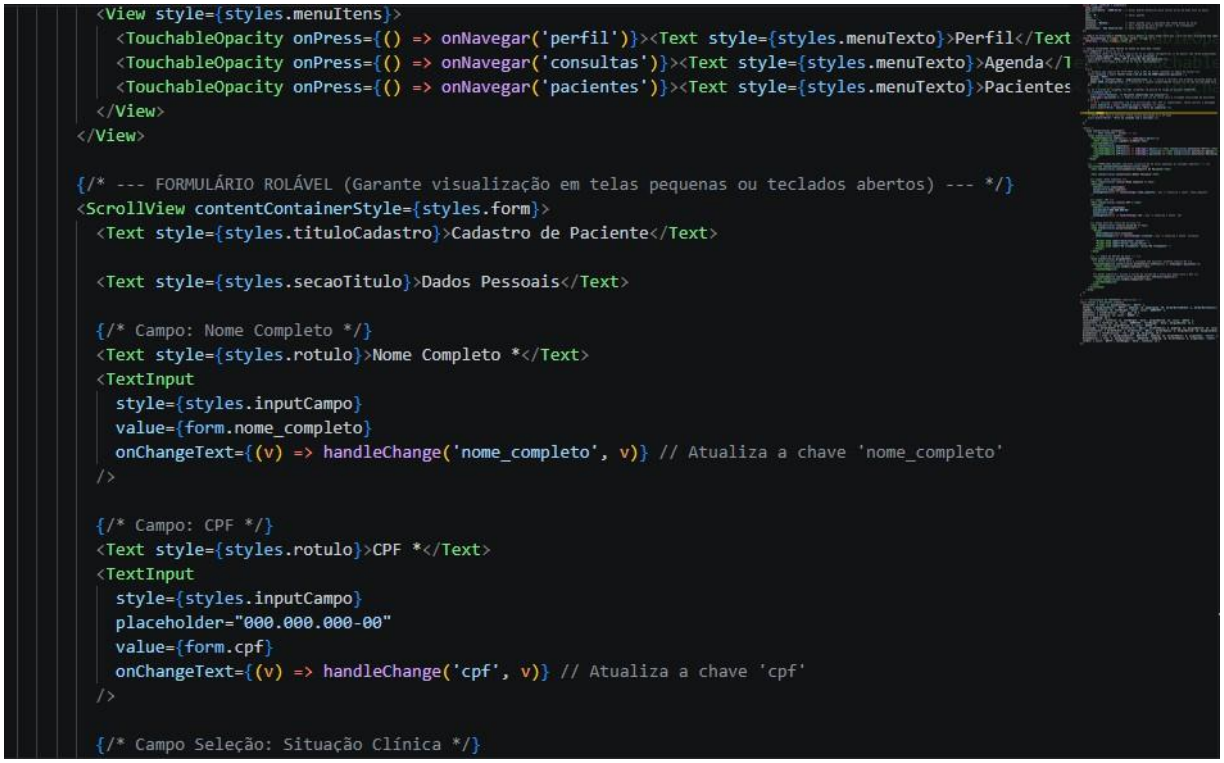


```
export default function CadastroPaciente({ onNavegar }: { onNavegar: (tela: string) => void }) {
  const cadastrar = async () => {

    try {
      // Dispara uma requisição HTTP POST para a API do Flask contendo os dados do formulário
      const resposta = await fetch('http://10.13.152.36:5000/cadastrar_paciente', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' }, // Avisa o servidor que estamos enviando dados em
        body: JSON.stringify(form) // Transforma o objeto JavaScript do formulário em uma String JSON pura
      });

      // Se o status da resposta for 201 (Created) ou dentro da faixa de sucesso (200-299)
      if (resposta.ok) {
        Alert.alert("Sucesso", "✓ Paciente cadastrado com sucesso!");
        onNavegar('pacientes'); // Redireciona o usuário de volta para a listagem atualizada de pacientes
      } else {
        // Se o servidor respondeu com erro estruturado (ex: CPF já cadastrado), tenta extrair a mensagem
        const dadosErro = await resposta.json().catch(() => ({}));
        Alert.alert("Erro", dadosErro.message || "Erro ao cadastrar.");
      }
    } catch (error) {
      // Entra aqui caso o servidor Flask esteja desligado ou o IP mude
      Alert.alert("Erro", "Erro na conexão com o servidor.");
    }
  };

  return (
    <View style={styles.container}>
      {/* --- MENU SUPERIOR / NAVBAR --- */}
      <View style={styles.navbar}>
        <TouchableOpacity onPress={() => onNavegar('perfil')}>
          <Text style={styles.logoNav}>CLINICA</Text>
```



```
      <View style={styles.menuItens}>
        <TouchableOpacity onPress={() => onNavegar('perfil')}><Text style={styles.menuTexto}>Perfil</Text
        <TouchableOpacity onPress={() => onNavegar('consultas')}><Text style={styles.menuTexto}>Agenda</T
        <TouchableOpacity onPress={() => onNavegar('pacientes')}><Text style={styles.menuTexto}>Pacientes
      </View>
    </View>

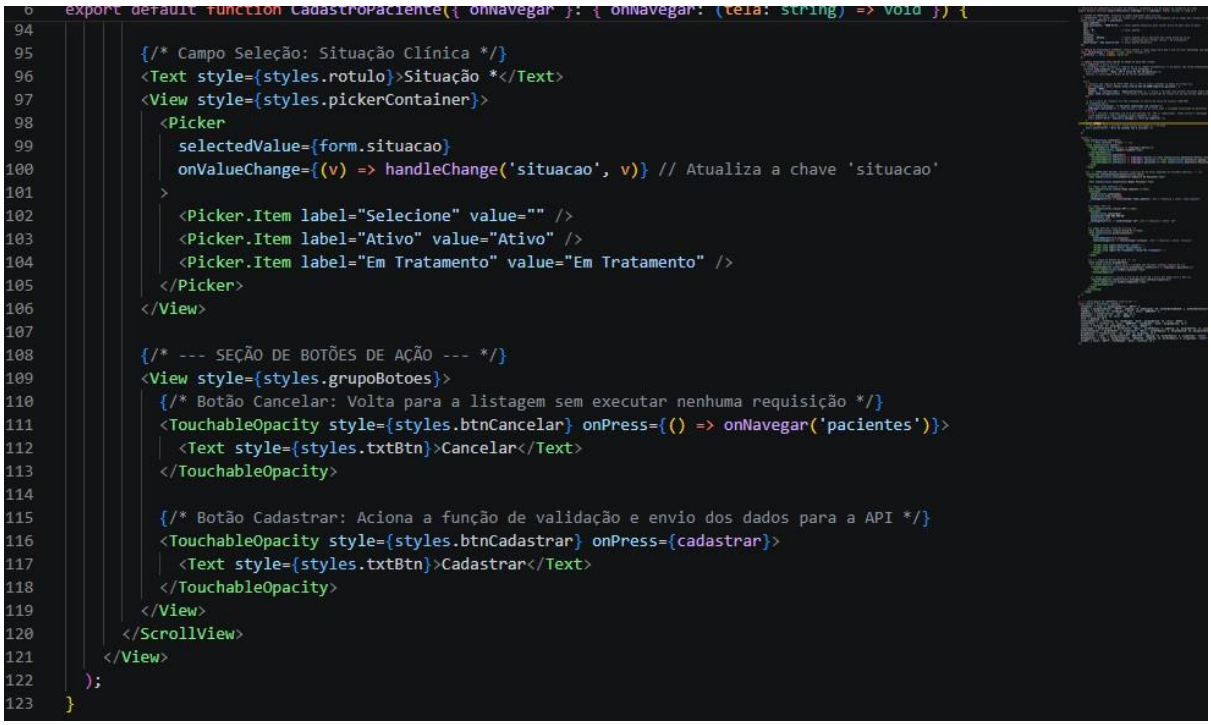
    {/* --- FORMULÁRIO ROLÁVEL (Garante visualização em telas pequenas ou teclados abertos) --- */}
    <ScrollView contentContainerStyle={styles.form}>
      <Text style={styles.tituloCadastro}>Cadastro de Paciente</Text>

      <Text style={styles.secaoTitulo}>Dados Pessoais</Text>

      {/* Campo: Nome Completo */}
      <Text style={styles.rotulo}>Nome Completo *</Text>
      <TextInput
        style={styles.inputCampo}
        value={form.nome_completo}
        onChangeText={(v) => handleChange('nome_completo', v)} // Atualiza a chave 'nome_completo'
      />

      {/* Campo: CPF */}
      <Text style={styles.rotulo}>CPF *</Text>
      <TextInput
        style={styles.inputCampo}
        placeholder="000.000.000-00"
        value={form.cpf}
        onChangeText={(v) => handleChange('cpf', v)} // Atualiza a chave 'cpf'
      />

      {/* Campo Seleção: Situação Clínica */}
```

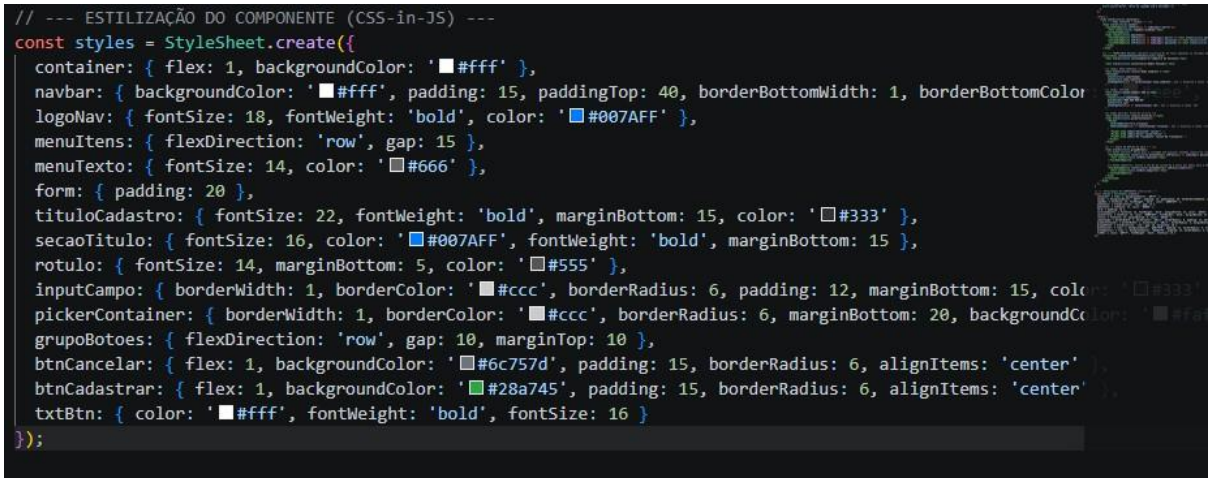


```
export default function CadastroPaciente({ onNavegar }: { onNavegar: (tela: string) => void }) {

        {/* Campo Seleção: Situação Clínica */}
        <Text style={styles.rotulo}>Situação *</Text>
        <View style={styles.pickerContainer}>
          <Picker
            selectedValue={form.situacao}
            onValueChange={(v) => handleChange('situacao', v)} // Atualiza a chave 'situacao'
          >
            <Picker.Item label="Selecione" value="" />
            <Picker.Item label="Ativo" value="Ativo" />
            <Picker.Item label="Em Tratamento" value="Em Tratamento" />
          </Picker>
        </View>

        {/* --- SEÇÃO DE BOTÕES DE AÇÃO --- */}
        <View style={styles.grupoBotoes}>
          {/* Botão Cancelar: Volta para a listagem sem executar nenhuma requisição */}
          <TouchableOpacity style={styles.btnCancelar} onPress={() => onNavegar('pacientes')}>
            <Text style={styles.txtBtn}>Cancelar</Text>
          </TouchableOpacity>

          {/* Botão Cadastrar: Aciona a função de validação e envio dos dados para a API */}
          <TouchableOpacity style={styles.btnCadastrar} onPress={cadastrar}>
            <Text style={styles.txtBtn}>Cadastrar</Text>
          </TouchableOpacity>
        </View>
      </ScrollView>
    </View>
  );
}
```



```
// --- ESTILIZAÇÃO DO COMPONENTE (CSS-in-JS) ---
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  navbar: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
  logoNav: { fontSize: 18, fontWeight: 'bold', color: '#007AFF' },
  menuItens: { flexDirection: 'row', gap: 15 },
  menuTexto: { fontSize: 14, color: '#666' },
  form: { padding: 20 },
  tituloCadastro: { fontSize: 22, fontWeight: 'bold', marginBottom: 15, color: '#333' },
  secaoTitulo: { fontSize: 16, color: '#007AFF', fontWeight: 'bold', marginBottom: 15 },
  rotulo: { fontSize: 14, marginBottom: 5, color: '#555' },
  inputCampo: { borderWidth: 1, borderColor: '#ccc', borderRadius: 6, padding: 12, marginBottom: 15, colo
  pickerContainer: { borderWidth: 1, borderColor: '#ccc', borderRadius: 6, marginBottom: 20, backgroundCo
  grupoBotoes: { flexDirection: 'row', gap: 10, marginTop: 10 },
  btnCancelar: { flex: 1, backgroundColor: '#6c757d', padding: 15, borderRadius: 6, alignItems: 'center'
  btnCadastrar: { flex: 1, backgroundColor: '#28a745', padding: 15, borderRadius: 6, alignItems: 'center'
  txtBtn: { color: '#fff', fontWeight: 'bold', fontSize: 16 }
});
```

Cadastrar estagiário:



```
Site-Principal > clinica-mobile > app > CadastroUsuario.tsx > ...
1  import React, { useState } from 'react';
2  import { View, Text, TextInput, TouchableOpacity, StyleSheet, ScrollView, Alert } from 'react-native';
3
4  // Definição do Componente de Cadastro de Usuário (Estagiários)
5  export default function CadastroUsuario({ onNavegar }: { onNavegar: (tela: string) => void }) {
6    // ESTADOS LOCAIS: Armazenam de forma separada os textos digitados nos inputs
7    const [nome, setNome] = useState('');
8    const [email, setEmail] = useState('');
9    const [senha, setSenha] = useState('');
10
11   // FUNÇÃO ASSÍNCRONA PARA ENVIO E CRIAÇÃO DA CONTA NO SERVIDOR
12   const criarConta = async () => {
13     // VALIDAÇÃO LOCAL SIMPLES: Garante que nenhum campo seja enviado em branco
14     if (!nome || !email || !senha) {
15       Alert.alert("Erro", "Por favor, preencha todos os campos.");
16       return; // Bloqueia a execução se faltar algo
17     }
18
19     try {
20       // Dispara uma requisição POST enviando os dados informados para a rota de cadastro de estagiários
21       const resposta = await fetch('http://10.13.152.36:5000/cadastrar_estagiario', {
22         method: 'POST',
23         headers: { 'Content-Type': 'application/json' }, // Define o formato do corpo como JSON
24         // NOTA: Injetamos valores fixos de segurança (cargo e data_nascimento) para preencher os campos ob
25         body: JSON.stringify({
26           nome,
27           email,
28           senha,
29           cargo: "Estagiário",
30           data_nascimento: "2000-01-01"
31         })
32       });
```

```
  const criarConta = async () => {
      });

      // Transforma a resposta do Flask em objeto JavaScript manipulável
      const dados = await resposta.json();

      // Se a resposta retornar sucesso (Status HTTP 201 Created)
      if (resposta.ok) {
        Alert.alert("Sucesso", "✓ Conta criada com sucesso!");
        onNavegar('login'); // Redireciona o usuário recém-criado para a tela de autenticação
      } else {
        // Exibe o erro retornado pela validação do back-end (ex: Email já cadastrado)
        Alert.alert("Erro", dados.message);
      }
    } catch (error) {
      // Executado caso o servidor Flask caia ou o dispositivo perca a conexão de rede
      Alert.alert("Erro", "Erro na conexão com o servidor.");
    }
  };

  return (
    // ScrollView com 'flexGrow: 1' garante o alinhamento centralizado sem cortar elementos em telas menore
    <ScrollView contentContainerStyle={styles.container}>
      {/* Identidade visual / Logo do sistema */}
      <Text style={styles.logo}>CLINICA</Text>
      <Text style={styles.titulo}>Cadastrar</Text>

      {/* Campo: Nome */}
      <Text style={styles.label}>Nome</Text>
      <TextInput
        style={styles.input}
```

```
export default function CadastroUsuario({ onNavegar }: { onNavegar: (tela: string) => void }) {
      />

      {/* Campo: E-mail */}
      <Text style={styles.label}>E-mail</Text>
      <TextInput
        style={styles.input}
        placeholder="exemplo@email.com"
        keyboardType="email-address" // Abre o teclado nativo otimizado para e-mails (com '@' e '.')
        autoCapitalize="none"        // Evita que o celular force a primeira letra maiúscula
        value={email}
        onChangeText={setEmail}
      />

      {/* Campo: Senha */}
      <Text style={styles.label}>Senha</Text>
      <TextInput
        style={styles.input}
        placeholder="Crie uma senha"
        secureTextEntry             // Oculta os caracteres digitados substituindo por pontos ocultos
        value={senha}
        onChangeText={setSenha}
      />

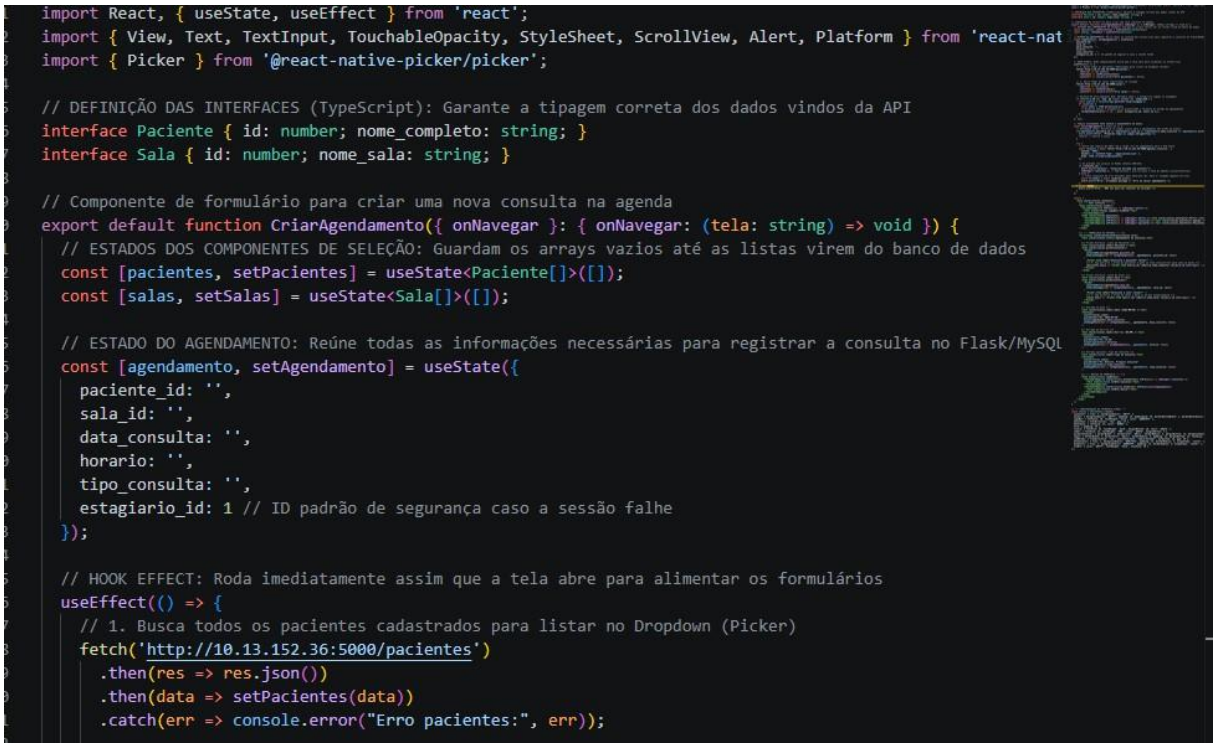
      {/* Botão de Envio do formulário */}
      <TouchableOpacity style={styles.btn} onPress={criarConta}>
        <Text style={styles.btnTexto}>Cadastrar</Text>
      </TouchableOpacity>

      {/* Link de Navegação Inferior para retornar ao login */}
      <TouchableOpacity style={styles.footer} onPress={() => onNavegar('login')}>
        <Text style={styles.footerTexto}>Já tem conta? <Text style={styles.link}>Voltar ao Login</Text></Te
```



```
// --- CONFIGURAÇÃO VISUAL DE DESIGN (CSS) ---
const styles = StyleSheet.create({
  // flexGrow permite que o contêiner se expanda de forma responsiva ao conteúdo interno
  container: { flexGrow: 1, padding: 20, backgroundColor: '#fff', justifyContent: 'center' },
  logo: { fontSize: 26, fontWeight: 'bold', textAlign: 'center', marginBottom: 30, color: '#007AFF' },
  titulo: { fontSize: 22, fontWeight: 'bold', marginBottom: 20 },
  label: { fontSize: 14, marginBottom: 5, color: '#555' },
  input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 6, padding: 12, marginBottom: 15 },
  btn: { backgroundColor: '#007AFF', padding: 15, borderRadius: 6, alignItems: 'center', marginTop: 10 },
  btnTexto: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
  footer: { marginTop: 25, alignItems: 'center' },
  footerTexto: { color: '#666' },
  link: { color: '#007AFF', textDecorationLine: 'underline' } // Sublinhado simulando link web
});
```

Criar agendamento:



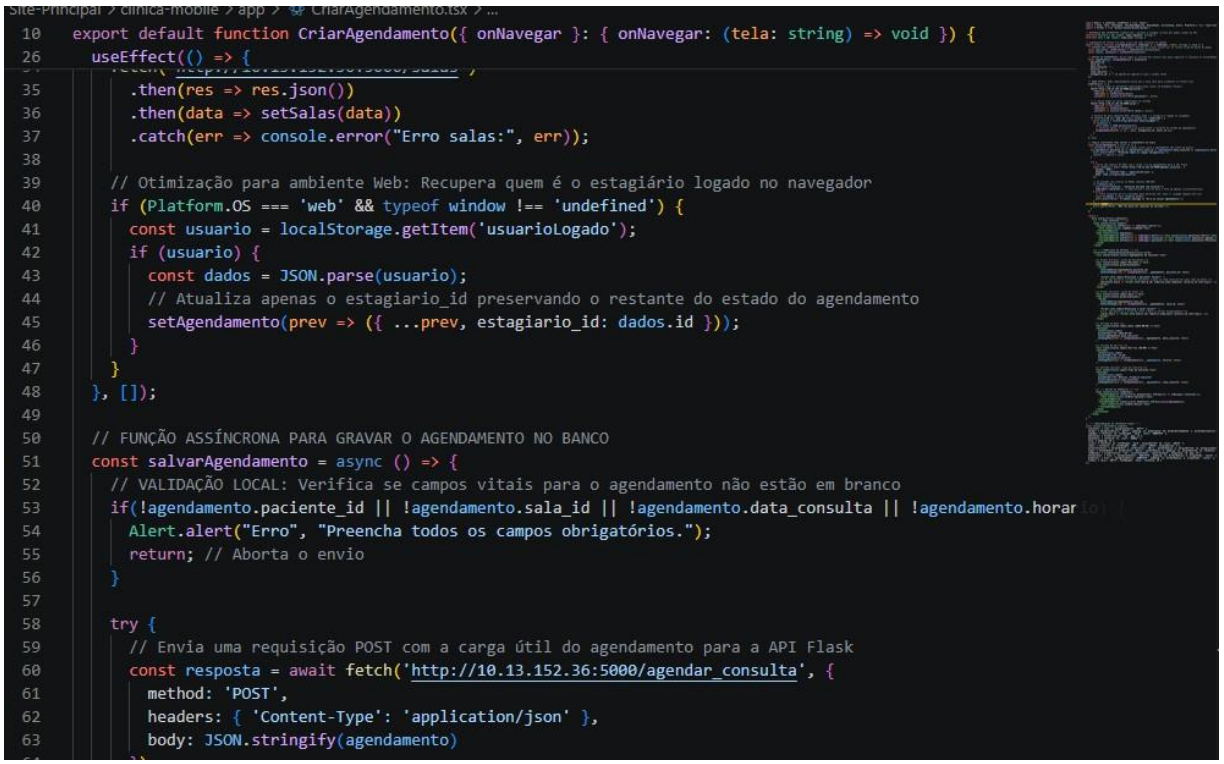
```
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, ScrollView, Alert, Platform } from 'react-nat
import { Picker } from '@react-native-picker/picker';

// DEFINIÇÃO DAS INTERFACES (TypeScript): Garante a tipagem correta dos dados vindos da API
interface Paciente { id: number; nome_completo: string; }
interface Sala { id: number; nome_sala: string; }

// Componente de formulário para criar uma nova consulta na agenda
export default function CriarAgendamento({ onNavegar }: { onNavegar: (tela: string) => void }) {
  // ESTADOS DOS COMPONENTES DE SELEÇÃO: Guardam os arrays vazios até as listas virem do banco de dados
  const [pacientes, setPacientes] = useState<Paciente[]>([]);
  const [salas, setSalas] = useState<Sala[]>([]);

  // ESTADO DO AGENDAMENTO: Reúne todas as informações necessárias para registrar a consulta no Flask/MySQL
  const [agendamento, setAgendamento] = useState({
    paciente_id: '',
    sala_id: '',
    data_consulta: '',
    horario: '',
    tipo_consulta: '',
    estagiario_id: 1 // ID padrão de segurança caso a sessão falhe
  });

  // HOOK EFFECT: Roda imediatamente assim que a tela abre para alimentar os formulários
  useEffect(() => {
    // 1. Busca todos os pacientes cadastrados para listar no Dropdown (Picker)
    fetch('http://10.13.152.36:5000/pacientes')
      .then(res => res.json())
      .then(data => setPacientes(data))
      .catch(err => console.error("Erro pacientes:", err));
```



```
export default function CriarAgendamento({ onNavegar }: { onNavegar: (tela: string) => void }) {
  useEffect(() => {
    .then(res => res.json())
    .then(data => setSalas(data))
    .catch(err => console.error("Erro salas:", err));

    // Otimização para ambiente Web: Recupera quem é o estagiário logado no navegador
    if (Platform.OS === 'web' && typeof window !== 'undefined') {
      const usuario = localStorage.getItem('usuarioLogado');
      if (usuario) {
        const dados = JSON.parse(usuario);
        // Atualiza apenas o estagiario_id preservando o restante do estado do agendamento
        setAgendamento(prev => ({ ...prev, estagiario_id: dados.id }));
      }
    }
  }, []);

  // FUNÇÃO ASSÍNCRONA PARA GRAVAR O AGENDAMENTO NO BANCO
  const salvarAgendamento = async () => {
    // VALIDAÇÃO LOCAL: Verifica se campos vitais para o agendamento não estão em branco
    if(!agendamento.paciente_id || !agendamento.sala_id || !agendamento.data_consulta || !agendamento.horar
      Alert.alert("Erro", "Preencha todos os campos obrigatórios.");
      return; // Aborta o envio
    }

    try {
      // Envia uma requisição POST com a carga útil do agendamento para a API Flask
      const resposta = await fetch('http://10.13.152.36:5000/agendar_consulta', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(agendamento)
```



```
export default function CriarAgendamento({ onNavegar }: { onNavegar: (tela: string) => void }) {
  const salvarAgendamento = async () => {
      const resposta = await fetch('http://10.13.152.36:5000/agendar_consulta', {
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(agendamento)
      });

      // Se gravado com sucesso no MySQL (Status 200/201)
      if (resposta.ok) {
        Alert.alert("Sucesso", "Consulta marcada com sucesso!");
        onNavegar('consultas'); // Redireciona o usuário para a tela da Agenda (ListarConsultas)
      } else {
        // Trata respostas de erro enviadas pelo back-end (Ex: Sala já ocupada naquele horário)
        const erroDados = await resposta.json();
        Alert.alert("Erro", erroDados.message || "Erro ao salvar agendamento.");
      }
    } catch (error) {
      Alert.alert("Erro", "Não foi possível conectar ao servidor.");
    }
  };

  return (
    <View style={styles.container}>
      {/* --- MENU SUPERIOR --- */}
      <View style={styles.navbar}>
        <TouchableOpacity onPress={() => onNavegar('perfil')}>
          <Text style={styles.logoNav}>CLINICA</Text>
        </TouchableOpacity>
        <View style={styles.menuItens}>
          <TouchableOpacity onPress={() => onNavegar('perfil')}><Text style={styles.menuTexto}>Perfil</Text
          <TouchableOpacity onPress={() => onNavegar('consultas')}><Text style={styles.menuTexto}>Agenda</T
          <TouchableOpacity onPress={() => onNavegar('pacientes')}><Text style={styles.menuTexto}>Pacientes
```

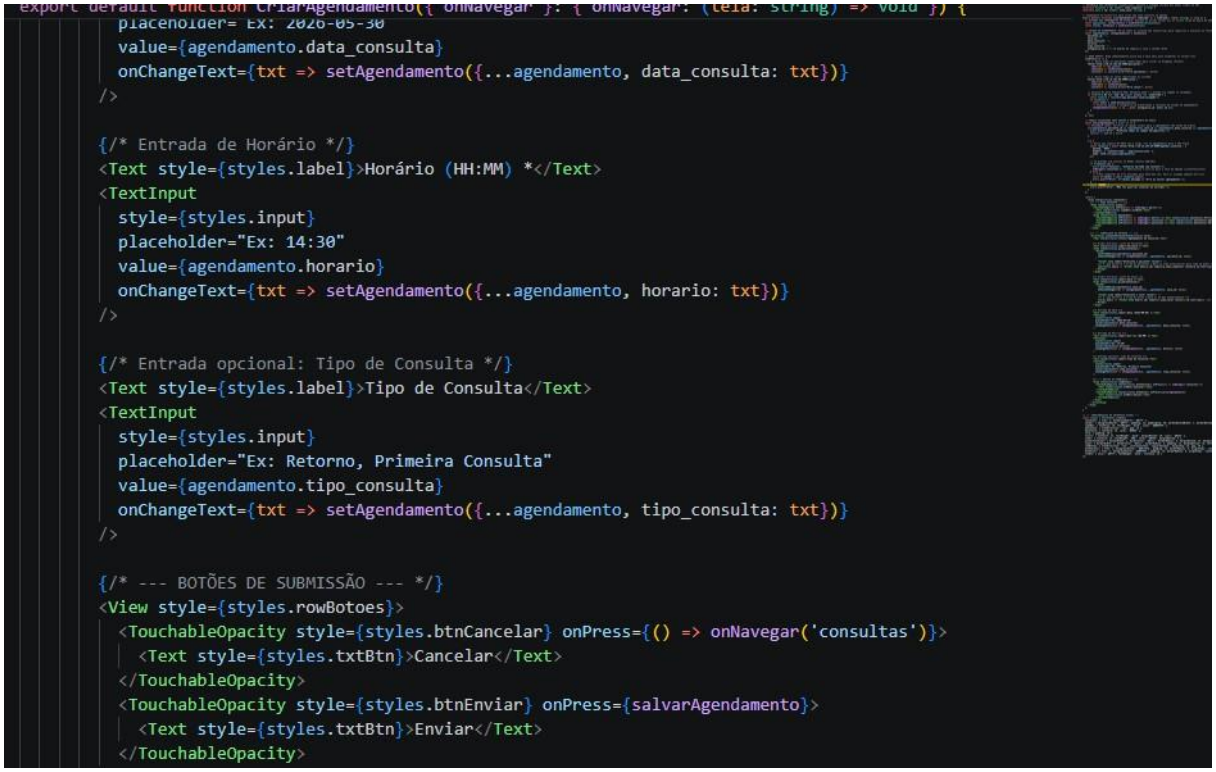


```
{/* --- FORMULÁRIO DE ENTRADA --- */}
<ScrollView contentContainerStyle={styles.form}>
  <Text style={styles.tituls}>Agendamento de Consulta</Text>

  {/* Picker Dinâmico: Lista de Pacientes */}
  <Text style={styles.label}>Paciente *</Text>
  <View style={styles.pickerContainer}>
    <Picker
      selectedValue={agendamento.paciente_id}
      onValueChange={val => setAgendamento({...agendamento, paciente_id: val})}
    >
      <Picker.Item label="Selecione o paciente" value="" />
      {/* O .map percorre o array de pacientes e gera um item selecionável para cada um deles */}
      {pacientes.map(p => <Picker.Item key={p.id} label={p.nome_completo} value={p.id.toString()} />)}
    </Picker>
  </View>

  {/* Picker Dinâmico: Lista de Salas */}
  <Text style={styles.label}>Sala *</Text>
  <View style={styles.pickerContainer}>
    <Picker
      selectedValue={agendamento.sala_id}
      onValueChange={val => setAgendamento({...agendamento, sala_id: val})}
    >
      <Picker.Item label="Selecione a sala" value="" />
      {/* O .map percorre o array de salas e gera as opções dinamicamente */}
      {salas.map(s => <Picker.Item key={s.id} label={s.nome_sala} value={s.id.toString()} />)}
    </Picker>
  </View>

  {/* Entrada de Data */}
```



```
    placeholder= Ex: 2026-05-30
    value={agendamento.data_consulta}
    onChangeText={txt => setAgendamento({...agendamento, data_consulta: txt})}
  />

  {/* Entrada de Horário */}
  <Text style={styles.label}>Horário (HH:MM) *</Text>
  <TextInput
    style={styles.input}
    placeholder="Ex: 14:30"
    value={agendamento.horario}
    onChangeText={txt => setAgendamento({...agendamento, horario: txt})}
  />

  {/* Entrada opcional: Tipo de Consulta */}
  <Text style={styles.label}>Tipo de Consulta</Text>
  <TextInput
    style={styles.input}
    placeholder="Ex: Retorno, Primeira Consulta"
    value={agendamento.tipo_consulta}
    onChangeText={txt => setAgendamento({...agendamento, tipo_consulta: txt})}
  />

  {/* --- BOTÕES DE SUBMISSÃO --- */}
  <View style={styles.rowBotoes}>
    <TouchableOpacity style={styles.btnCancelar} onPress={() => onNavegar('consultas')}>
      <Text style={styles.txtBtn}>Cancelar</Text>
    </TouchableOpacity>
    <TouchableOpacity style={styles.btnEnviar} onPress={salvarAgendamento}>
      <Text style={styles.txtBtn}>Enviar</Text>
    </TouchableOpacity>
```

```
    </View>
  );
}

// --- CONFIGURAÇÕES DE INTERFACE VISUAL ---
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  navbar: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
  logoNav: { fontSize: 18, fontWeight: 'bold', color: '#007AFF' },
  menuItens: { flexDirection: 'row', gap: 15 },
  menuTexto: { fontSize: 14, color: '#666' },
  form: { padding: 20 },
  tituls: { fontSize: 22, fontWeight: 'bold', marginBottom: 20, color: '#333' },
  label: { fontSize: 14, fontWeight: '500', color: '#555', marginBottom: 5 },
  pickerContainer: { borderWidth: 1, borderColor: '#ccc', borderRadius: 6, marginBottom: 15, backgroundCo
  input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 6, padding: 12, marginBottom: 15, fontSize:
  rowBotoes: { flexDirection: 'row', justifyContent: 'space-between', marginTop: 15, gap: 10 },
  btnCancelar: { flex: 1, backgroundColor: '#6c757d', padding: 15, borderRadius: 6, alignItems: 'center'
  btnEnviar: { flex: 1, backgroundColor: '#007AFF', padding: 15, borderRadius: 6, alignItems: 'center' },
  txtBtn: { color: '#fff', fontWeight: 'bold', fontSize: 16 }
});
```

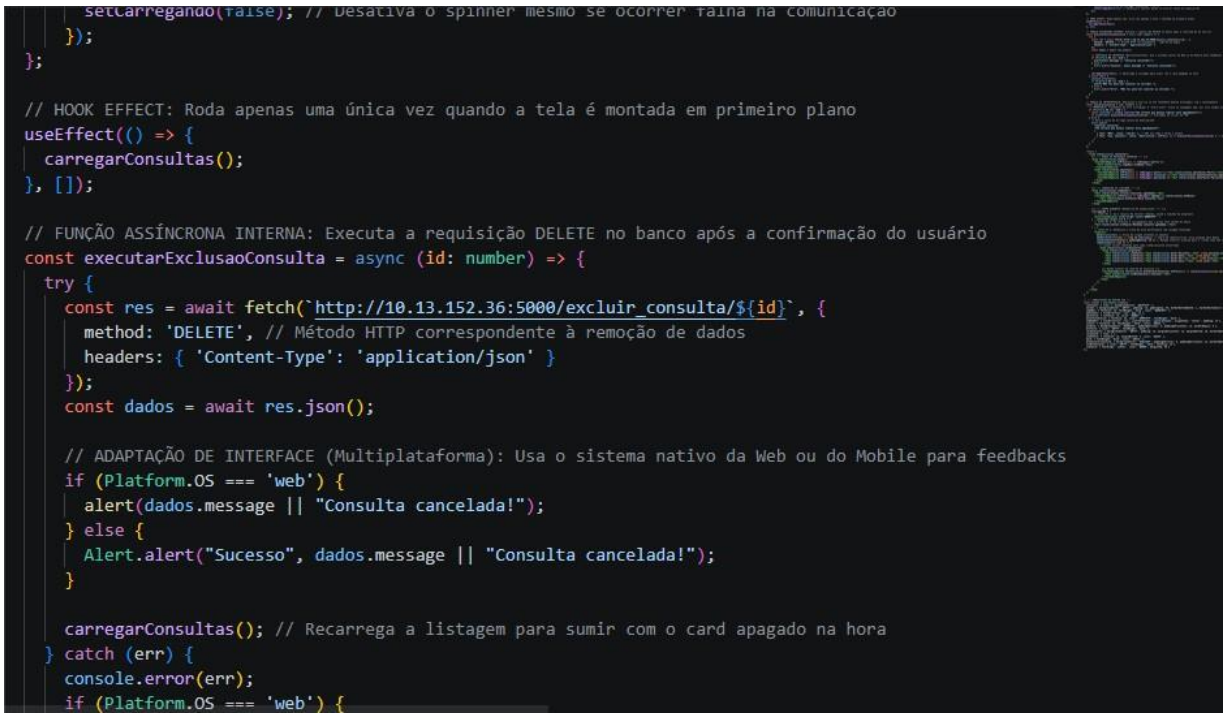
Lista de consulta:

```
import React, { useEffect, useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet, FlatList, ActivityIndicator, Alert, Platform } from 'rea

// INTERFACE (TypeScript): Define a estrutura de dados esperada para cada consulta vinda da API
interface Consulta {
  id: number;
  paciente_nome: string;
  data: string;
  horario: string;
  sala: string;
}

// Componente encarregado de listar todos os agendamentos da clínica medica
export default function ListarConsultas({ onNavegar }: { onNavegar: (tela: string) => void }) {
  // ESTADOS DO COMPONENTE
  const [consultas, setConsultas] = useState<Consulta[]>([]); // Guarda o array de consultas retornado pela
  const [carregando, setCarregando] = useState(true);          // Controla o indicador visual de carregamer

  // FUNÇÃO REUTILIZÁVEL: Faz uma requisição GET para puxar a lista de consultas atualizada do servidor Fla
  const carregarConsultas = () => {
    setCarregando(true); // Ativa o spinner antes de disparar o fetch
    fetch('http://10.13.152.36:5000/consultas')
      .then(res => res.json()) // Converte a resposta do Flask em JSON estruturado
      .then(dados => {
        setConsultas(dados);  // Salva os dados recebidos no estado
        setCarregando(false); // Desativa o spinner de carregamento
      })
      .catch(err => {
        console.error("Erro ao carregar consultas:", err);
        setCarregando(false); // Desativa o spinner mesmo se ocorrer falha na comunicação
      });
  };
```



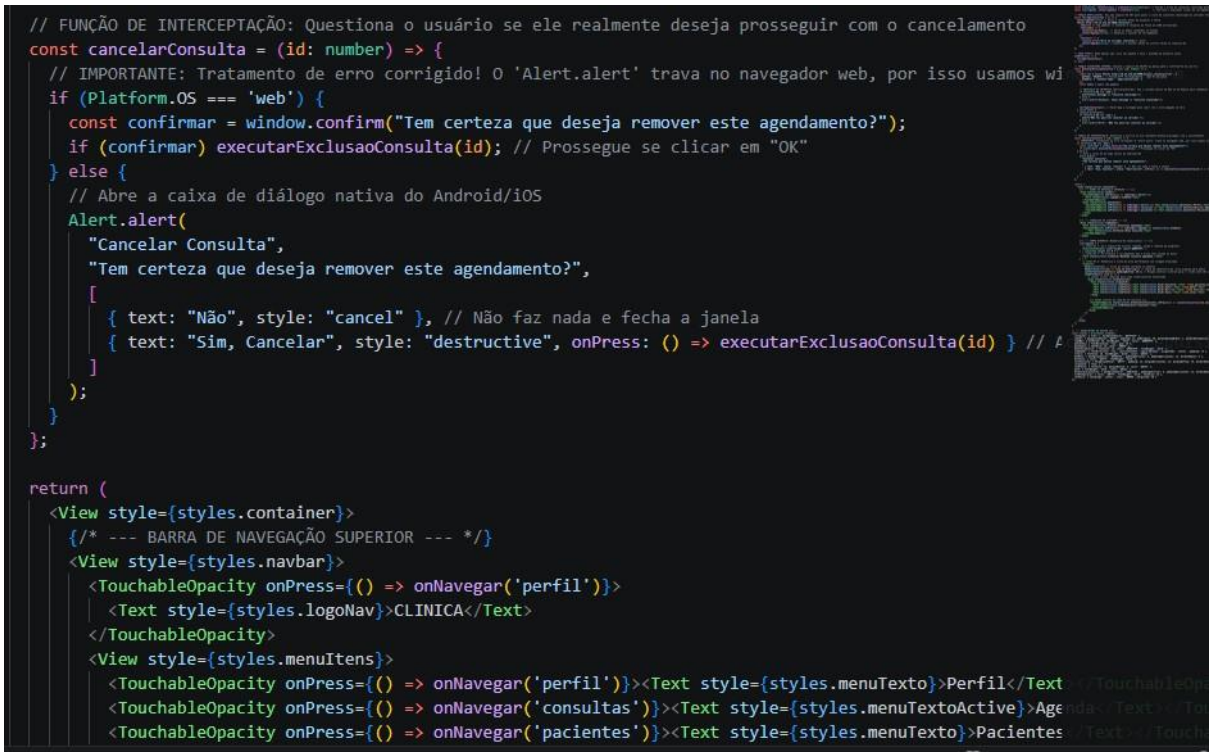
```
      setCarregando(false); // Desativa o spinner mesmo se ocorrer falha na comunicação
    });
};

// HOOK EFFECT: Roda apenas uma única vez quando a tela é montada em primeiro plano
useEffect(() => {
  carregarConsultas();
}, []);

// FUNÇÃO ASSÍNCRONA INTERNA: Executa a requisição DELETE no banco após a confirmação do usuário
const executarExclusaoConsulta = async (id: number) => {
  try {
    const res = await fetch(`http://10.13.152.36:5000/excluir_consulta/${id}`, {
      method: 'DELETE', // Método HTTP correspondente à remoção de dados
      headers: { 'Content-Type': 'application/json' }
    });
    const dados = await res.json();

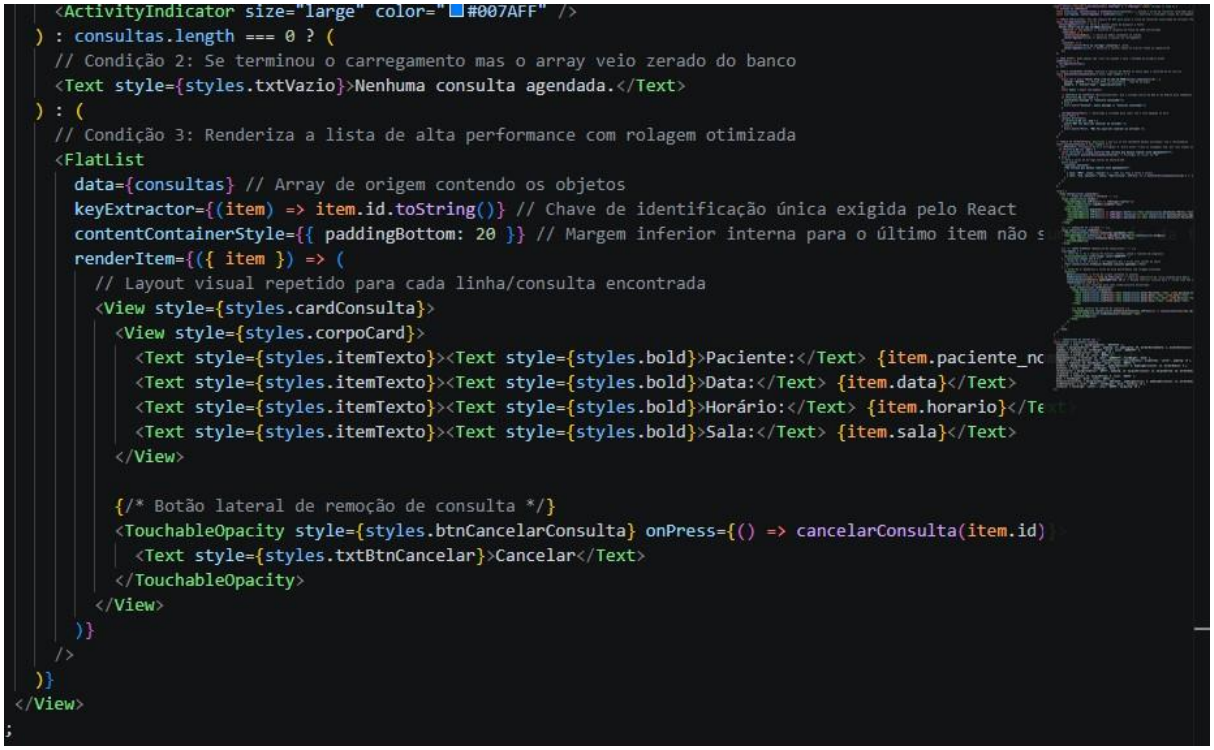
    // ADAPTAÇÃO DE INTERFACE (Multiplataforma): Usa o sistema nativo da Web ou do Mobile para feedbacks
    if (Platform.OS === 'web') {
      alert(dados.message || "Consulta cancelada!");
    } else {
      Alert.alert("Sucesso", dados.message || "Consulta cancelada!");
    }

    carregarConsultas(); // Recarrega a listagem para sumir com o card apagado na hora
  } catch (err) {
    console.error(err);
    if (Platform.OS === 'web') {
```



```
// FUNÇÃO DE INTERCEPTAÇÃO: Questiona o usuário se ele realmente deseja prosseguir com o cancelamento
const cancelarConsulta = (id: number) => {
  // IMPORTANTE: Tratamento de erro corrigido! O 'Alert.alert' trava no navegador web, por isso usamos wi
  if (Platform.OS === 'web') {
    const confirmar = window.confirm("Tem certeza que deseja remover este agendamento?");
    if (confirmar) executarExclusaoConsulta(id); // Prossegue se clicar em "OK"
  } else {
    // Abre a caixa de diálogo nativa do Android/iOS
    Alert.alert(
      "Cancelar Consulta",
      "Tem certeza que deseja remover este agendamento?",
      [
        { text: "Não", style: "cancel" }, // Não faz nada e fecha a janela
        { text: "Sim, Cancelar", style: "destructive", onPress: () => executarExclusaoConsulta(id) } // A
      ]
    );
  }
};

return (
  <View style={styles.container}>
    {/* --- BARRA DE NAVEGAÇÃO SUPERIOR --- */}
    <View style={styles.navbar}>
      <TouchableOpacity onPress={() => onNavegar('perfil')}>
        <Text style={styles.logoNav}>CLINICA</Text>
      </TouchableOpacity>
      <View style={styles.menuItens}>
        <TouchableOpacity onPress={() => onNavegar('perfil')}><Text style={styles.menuTexto}>Perfil</Text
        <TouchableOpacity onPress={() => onNavegar('consultas')}><Text style={styles.menuTextoActive}>Age
        <TouchableOpacity onPress={() => onNavegar('pacientes')}><Text style={styles.menuTexto}>Pacientes
```



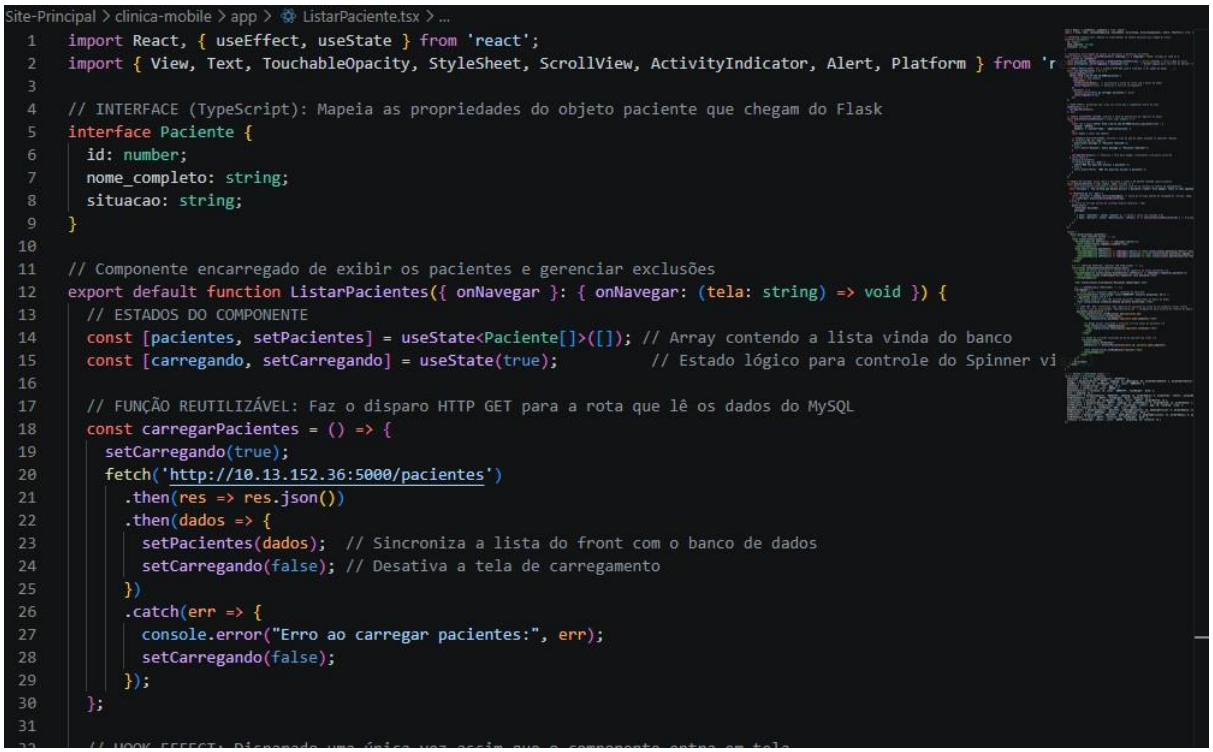
```
    <ActivityIndicator size="large" color="#007AFF" />
  ) : consultas.length === 0 ? (
    // Condição 2: Se terminou o carregamento mas o array veio zerado do banco
    <Text style={styles.txtVazio}>Nenhuma consulta agendada.</Text>
  ) : (
    // Condição 3: Renderiza a lista de alta performance com rolagem otimizada
    <FlatList
      data={consultas} // Array de origem contendo os objetos
      keyExtractor={(item) => item.id.toString()} // Chave de identificação única exigida pelo React
      contentContainerStyle={{ paddingBottom: 20 }} // Margem inferior interna para o último item não s
      renderItem={({ item }) => (
        // Layout visual repetido para cada linha/consulta encontrada
        <View style={styles.cardConsulta}>
          <View style={styles.corpoCard}>
            <Text style={styles.itemTexto}><Text style={styles.bold}>Paciente:</Text> {item.paciente_nc
            <Text style={styles.itemTexto}><Text style={styles.bold}>Data:</Text> {item.data}</Text>
            <Text style={styles.itemTexto}><Text style={styles.bold}>Horário:</Text> {item.horario}</Te
            <Text style={styles.itemTexto}><Text style={styles.bold}>Sala:</Text> {item.sala}</Text>
          </View>

          {/* Botão lateral de remoção de consulta */}
          <TouchableOpacity style={styles.btnCancelarConsulta} onPress={() => cancelarConsulta(item.id)
            <Text style={styles.txtBtnCancelar}>Cancelar</Text>
          </TouchableOpacity>
        </View>
      )}
    />
  )}
</View>
;
```



```
// --- ARQUITETURA DE DESIGN CSS ---
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  navbar: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
  logoNav: { fontSize: 18, fontWeight: 'bold', color: '#007AFF' },
  menuItens: { flexDirection: 'row', gap: 15 },
  menuTexto: { fontSize: 14, color: '#666' },
  menuTextoActive: { fontSize: 14, color: '#007AFF', fontWeight: 'bold' },
  topHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', padding: 15 },
  titulo: { fontSize: 18, fontWeight: 'bold', color: '#333' },
  btnNova: { backgroundColor: '#28a745', paddingVertical: 8, paddingHorizontal: 12, borderRadius: 6 },
  btnTexto: { color: '#fff', fontWeight: 'bold' },
  cardConsulta: { backgroundColor: '#fff', padding: 15, marginHorizontal: 15, marginBottom: 10, borderRad
  corpoCard: { flex: 1 },
  itemTexto: { fontSize: 15, marginBottom: 4, color: '#444' },
  bold: { fontWeight: 'bold', color: '#222' },
  btnCancelarConsulta: { backgroundColor: '#dc3545', paddingVertical: 8, paddingHorizontal: 12, borderRad
  txtBtnCancelar: { color: '#fff', fontWeight: 'bold', fontSize: 13 },
  txtVazio: { textAlign: 'center', color: '#999', marginTop: 40 }
});
```

Lista de paciente:

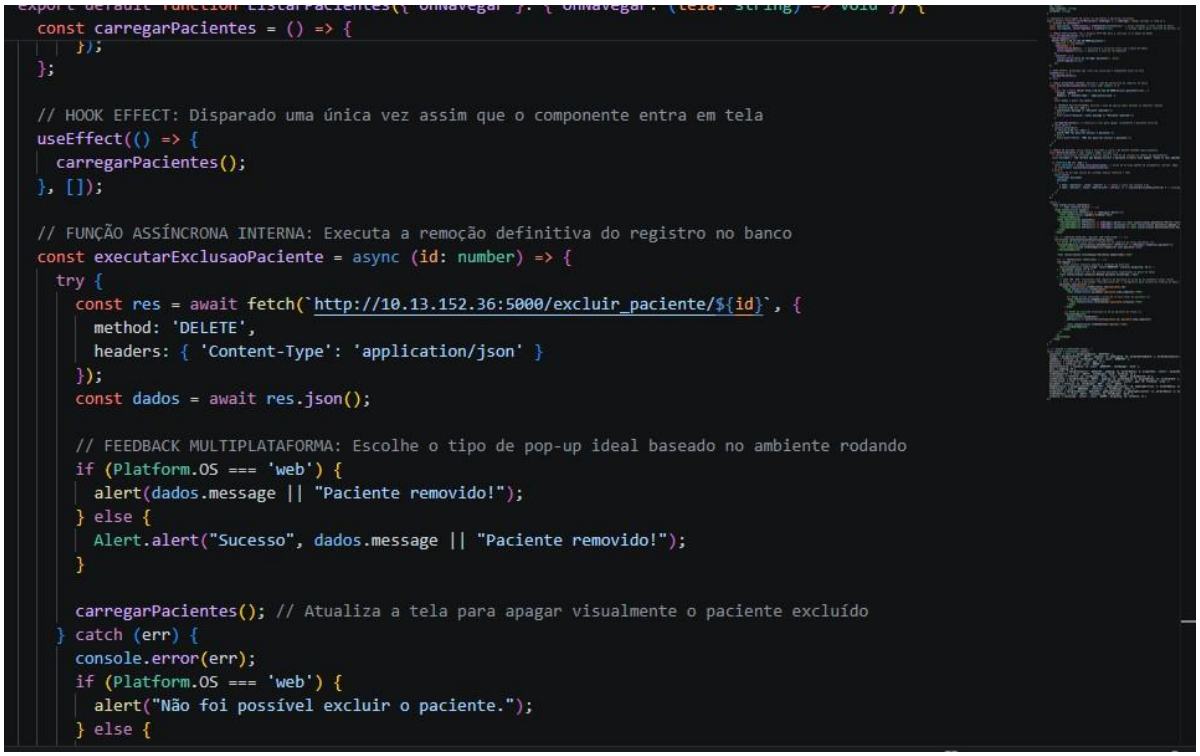


```
Site-Principal > clinica-mobile > app > ListarPaciente.tsx > ...
import React, { useEffect, useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet, ScrollView, ActivityIndicator, Alert, Platform } from 'r

// INTERFACE (TypeScript): Mapeia as propriedades do objeto paciente que chegam do Flask
interface Paciente {
  id: number;
  nome_completo: string;
  situacao: string;
}

// Componente encarregado de exibir os pacientes e gerenciar exclusões
export default function ListarPacientes({ onNavegar }: { onNavegar: (tela: string) => void }) {
  // ESTADOS DO COMPONENTE
  const [pacientes, setPacientes] = useState<Paciente[]>([]); // Array contendo a lista vinda do banco
  const [carregando, setCarregando] = useState(true);          // Estado lógico para controle do Spinner vi

  // FUNÇÃO REUTILIZÁVEL: Faz o disparo HTTP GET para a rota que lê os dados do MySQL
  const carregarPacientes = () => {
    setCarregando(true);
    fetch('http://10.13.152.36:5000/pacientes')
      .then(res => res.json())
      .then(dados => {
        setPacientes(dados);  // Sincroniza a lista do front com o banco de dados
        setCarregando(false); // Desativa a tela de carregamento
      })
      .catch(err => {
        console.error("Erro ao carregar pacientes:", err);
        setCarregando(false);
      });
  };
```



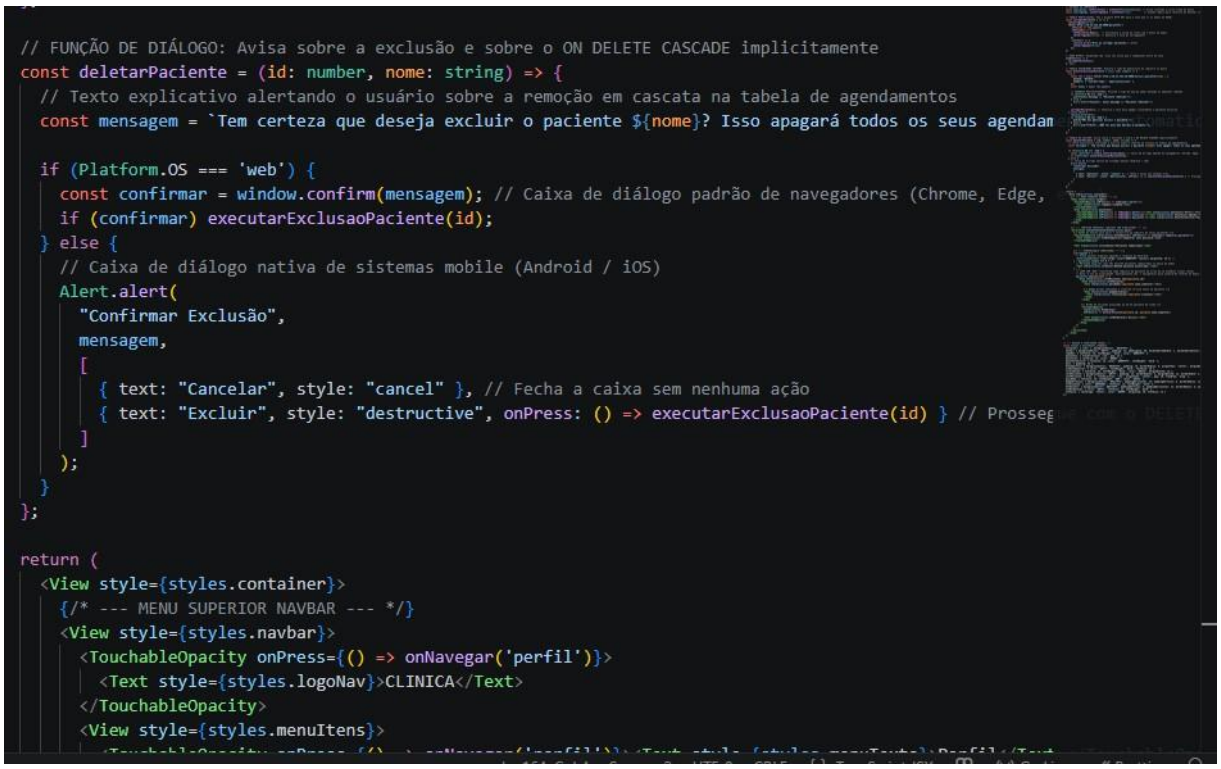
```
  const carregarPacientes = () => {
    });
};

// HOOK EFFECT: Disparado uma única vez assim que o componente entra em tela
useEffect(() => {
  carregarPacientes();
}, []);

// FUNÇÃO ASSÍNCRONA INTERNA: Executa a remoção definitiva do registro no banco
const executarExclusaoPaciente = async (id: number) => {
  try {
    const res = await fetch(`http://10.13.152.36:5000/excluir_paciente/${id}`, {
      method: 'DELETE',
      headers: { 'Content-Type': 'application/json' }
    });
    const dados = await res.json();

    // FEEDBACK MULTIPLATAFORMA: Escolhe o tipo de pop-up ideal baseado no ambiente rodando
    if (Platform.OS === 'web') {
      alert(dados.message || "Paciente removido!");
    } else {
      Alert.alert("Sucesso", dados.message || "Paciente removido!");
    }

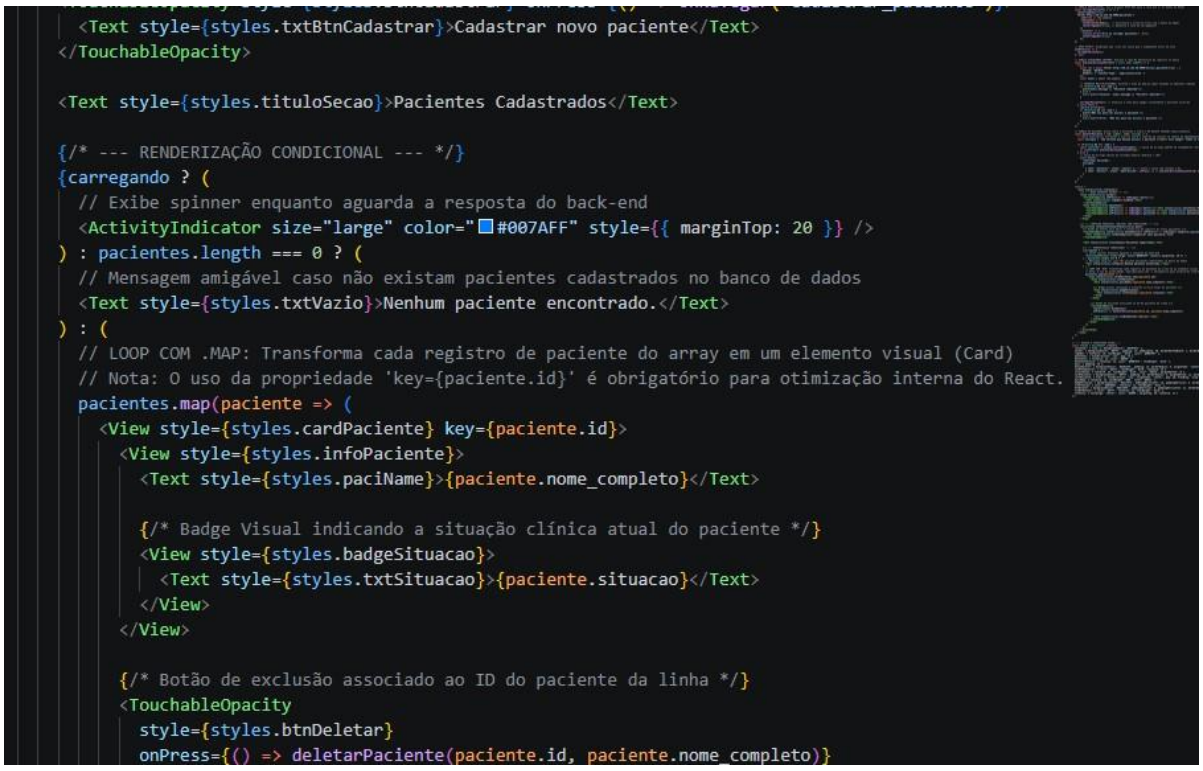
    carregarPacientes(); // Atualiza a tela para apagar visualmente o paciente excluído
  } catch (err) {
    console.error(err);
    if (Platform.OS === 'web') {
      alert("Não foi possível excluir o paciente.");
    } else {
```



```
// FUNÇÃO DE DIÁLOGO: Avisa sobre a exclusão e sobre o ON DELETE CASCADE implicitamente
const deletarPaciente = (id: number, nome: string) => {
  // Texto explicativo alertando o usuário sobre a remoção em cascata na tabela de agendamentos
  const mensagem = `Tem certeza que deseja excluir o paciente ${nome}? Isso apagará todos os seus agendam

  if (Platform.OS === 'web') {
    const confirmar = window.confirm(mensagem); // Caixa de diálogo padrão de navegadores (Chrome, Edge,
    if (confirmar) executarExclusaoPaciente(id);
  } else {
    // Caixa de diálogo nativa de sistemas mobile (Android / iOS)
    Alert.alert(
      "Confirmar Exclusão",
      mensagem,
      [
        { text: "Cancelar", style: "cancel" }, // Fecha a caixa sem nenhuma ação
        { text: "Excluir", style: "destructive", onPress: () => executarExclusaoPaciente(id) } // Prosseg
      ]
    );
  }
};

return (
  <View style={styles.container}>
    {/* --- MENU SUPERIOR NAVBAR --- */}
    <View style={styles.navbar}>
      <TouchableOpacity onPress={() => onNavegar('perfil')}>
        <Text style={styles.logoNav}>CLINICA</Text>
      </TouchableOpacity>
      <View style={styles.menuItens}>
```



```
      <Text style={styles.txtBtnCadastrar}>Cadastrar novo paciente</Text>
    </TouchableOpacity>

    <Text style={styles.tituloSecao}>Pacientes Cadastrados</Text>

    {/* --- RENDERIZAÇÃO CONDICIONAL --- */}
    {carregando ? (
      // Exibe spinner enquanto aguarda a resposta do back-end
      <ActivityIndicator size="large" color="#007AFF" style={{ marginTop: 20 }} />
    ) : pacientes.length === 0 ? (
      // Mensagem amigável caso não existam pacientes cadastrados no banco de dados
      <Text style={styles.txtVazio}>Nenhum paciente encontrado.</Text>
    ) : (
      // LOOP COM .MAP: Transforma cada registro de paciente do array em um elemento visual (Card)
      // Nota: O uso da propriedade 'key={paciente.id}' é obrigatório para otimização interna do React.
      pacientes.map(paciente => (
        <View style={styles.cardPaciente} key={paciente.id}>
          <View style={styles.infoPaciente}>
            <Text style={styles.paciName}>{paciente.nome_completo}</Text>

            {/* Badge Visual indicando a situação clínica atual do paciente */}
            <View style={styles.badgeSituacao}>
              <Text style={styles.txtSituacao}>{paciente.situacao}</Text>
            </View>
          </View>

          {/* Botão de exclusão associado ao ID do paciente da linha */}
          <TouchableOpacity
            style={styles.btnDeletar}
            onPress={() => deletarPaciente(paciente.id, paciente.nome_completo)}
```

```
}

// --- DESIGN E IDENTIDADE VISUAL ---
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f8f9fa' },
  navbar: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
  logoNav: { fontSize: 18, fontWeight: 'bold', color: '#007AFF' },
  menuItens: { flexDirection: 'row', gap: 15 },
  menuTexto: { fontSize: 14, color: '#666' },
  menuTextoActive: { fontSize: 14, color: '#007AFF', fontWeight: 'bold' },
  main: { padding: 20 },
  btnCadastrar: { backgroundColor: '#28a745', padding: 15, borderRadius: 8, alignItems: 'center', marginB
  txtBtnCadastrar: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
  tituloSecao: { fontSize: 18, fontWeight: 'bold', color: '#333', marginBottom: 15 },
  cardPaciente: { backgroundColor: '#fff', padding: 15, borderRadius: 8, marginBottom: 12, borderWidth: 1
  infoPaciente: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 10, flexWrap: 'wrap' },
  paciName: { fontSize: 16, fontWeight: '600', color: '#333' },
  badgeSituacao: { backgroundColor: '#e1f5fe', paddingHorizontal: 10, paddingVertical: 5, borderRadius: 12
  txtSituacao: { color: '#0288d1', fontSize: 12, fontWeight: 'bold' },
  btnDeletar: { backgroundColor: '#dc3545', paddingVertical: 6, paddingHorizontal: 12, borderRadius: 6, m
  txtBtnDeletar: { color: '#fff', fontSize: 12, fontWeight: 'bold' },
  txtVazio: { textAlign: 'center', color: '#999', marginTop: 20, fontSize: 14 }
});
```

Login:

```
import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, Alert } from 'react-native';

// Definição do Componente de Login, recebendo a função global de navegação por abas
export default function Login({ onNavegar }: { onNavegar: (tela: string) => void }) {
  // ESTADOS LOCAIS: Armazenam as credenciais inseridas pelo usuário antes do envio
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  // FUNÇÃO ASSÍNCRONA PARA AUTENTICAÇÃO JUNTO AO FLASK
  const fazerLogin = async () => {
    // VALIDAÇÃO LOCAL PREVENTIVA: Impede requisições desnecessárias caso falte algum dado
    if (!email || !senha) {
      Alert.alert("Aviso", "Por favor, preencha todos os campos.");
      return; // Aborta a execução da função imediatamente
    }

    try {
      // Dispara uma requisição POST enviando e-mail e senha para validação no back-end
      const resposta = await fetch('http://10.13.152.36:5000/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' }, // Indica formato JSON
        body: JSON.stringify({ email, senha }) // Serializa as credenciais em string JSON
      });

      // Transforma o fluxo de resposta da API em objeto manipulável
      const dados = await resposta.json();

      // Se as credenciais baterem no MySQL (Status HTTP 200 OK)
      if (resposta.ok) {
        Alert.alert("Sucesso", "Login realizado com sucesso!");
```

```
    });

    // Transforma o fluxo de resposta da API em objeto manipulável
    const dados = await resposta.json();

    // Se as credenciais baterem no MySQL (Status HTTP 200 OK)
    if (resposta.ok) {
      Alert.alert("Sucesso", "Login realizado com sucesso!");

      // PERSISTÊNCIA DA SESSÃO (Web): Se estiver rodando no navegador, salva os dados do estagiário
      if (typeof window !== 'undefined') {
        localStorage.setItem('usuarioLogado', JSON.stringify(dados.estagiario));
      }

      // Navega o usuário autenticado com sucesso para a tela de Perfil/Principal
      onNavegar('perfil');
    } else {
      // Exibe a mensagem de falha estruturada pelo back-end (ex: "Senha inválida" ou usuário inexistente
      Alert.alert("Erro", dados.message || "E-mail ou senha incorretos.");
    }
  } catch (error) {
    // Entra em ação se a API Flask estiver fora do ar ou o IP da máquina mudar
    Alert.alert("Erro", "Não foi possível conectar ao servidor Flask.");
  }
};

return (
  <View style={styles.container}>
    {/* --- CABEÇALHO/LOGO SUPERIOR --- */}
    <View style={styles.header}>
```

```
    {/* --- ÁREA DO FORMULÁRIO CENTRALIZADO --- */}
    <View style={styles.main}>
      <Text style={styles.titulo}>Login</Text>

      {/* Input de E-mail */}
      <View style={styles.inputGrupo}>
        <Text style={styles.label}>Usuário (E-mail)</Text>
        <TextInput
          style={styles.input}
          placeholder="Digite seu email"
          placeholderTextColor="#999"
          value={email}
          onChangeText={(txt) => setEmail(txt)}
          keyboardType="email-address" // Configura o teclado mobile nativo com atalhos de e-mail ('@' e
          autoCapitalize="none"        // Evita letras maiúsculas automáticas inseridas pelo sistema mobi
        />
      </View>

      {/* Input de Senha */}
      <View style={styles.inputGrupo}>
        <Text style={styles.label}>Senha</Text>
        <TextInput
          style={styles.input}
          placeholder="Digite sua senha"
          placeholderTextColor="#999"
          secureTextEntry={true} // Oculta o texto substituindo-o por caracteres ocultos de segurança
          value={senha}
          onChangeText={(txt) => setSenha(txt)}
        />
      </View>
```

```
          <Text style={styles.txtCadastro}>
            Não tem uma conta?{' '}
            <Text
              style={styles.linkCadastro}
              onPress={() => onNavegar('cadastro_usuario')} // Aciona o roteador central para abrir o cadastr
            >
              Cadastre-se aqui
            </Text>
          </Text>
        </View>
      </View>
    );
  }

  // --- DESIGN VISUAL (CSS-IN-JS) ---
  const styles = StyleSheet.create({
    container: { flex: 1, backgroundColor: '#fff' },
    header: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
    logo: { fontSize: 20, fontWeight: 'bold', color: '#007AFF', letterSpacing: 1 },
    main: { flex: 1, padding: 25, justifyContent: 'center' }, // Centraliza verticalmente o formulário de log
    titulo: { fontSize: 28, fontWeight: 'bold', color: '#333', marginBottom: 30, textAlign: 'center' },
    inputGrupo: { marginBottom: 20 },
    label: { fontSize: 14, color: '#555', marginBottom: 8, fontWeight: '500' },
    input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8, padding: 12, fontSize: 16, backgroundCol
    btnLogin: { backgroundColor: '#007AFF', padding: 15, borderRadius: 8, alignItems: 'center', marginTop:
    txtBtnLogin: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
    txtCadastro: { marginTop: 25, textAlign: 'center', fontSize: 14, color: '#666' },
    linkCadastro: { color: '#007AFF', fontWeight: 'bold', textDecorationLine: 'underline' }
  });
```

Tela principal:

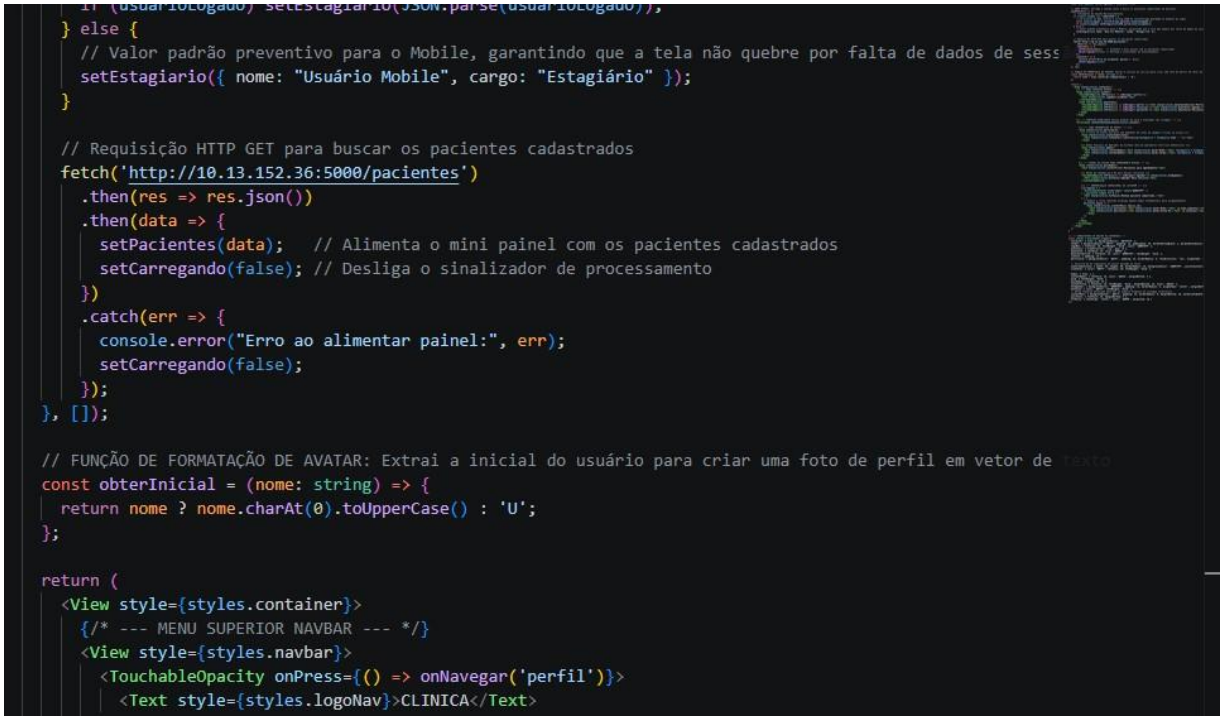
```
import React, { useEffect, useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet, ActivityIndicator, ScrollView } from 'react-native';

// INTERFACES (TypeScript): Documentam as estruturas de dados utilizadas na tela
interface Estagiario {
  nome: string;
  cargo: string;
}

interface Paciente {
  id: number;
  nome_completo: string;
  situacao: string;
}

// Componente da Tela Principal que serve como painel de controle do estagiário logado
export default function TelaPrincipal({ onNavegar }: { onNavegar: (tela: string) => void }) {
  // ESTADOS LOCAIS: O estagiário inicia como 'null' enquanto lê os dados locais da sessão
  const [estagiario, setEstagiario] = useState<Estagiario | null>(null);
  const [pacientes, setPacientes] = useState<Paciente[]>([]); // Lista simplificada de pacientes para agenc
  const [carregando, setCarregando] = useState(true);

  // HOOK EFFECT: Carrega a sessão local e busca os pacientes cadastrados em paralelo
  useEffect(() => {
    // Tratamento de Sessão Multiplataforma
    if (typeof window !== 'undefined') {
      // Se estiver na Web, extrai a string JSON do localStorage guardada no momento do Login
      const usuarioLogado = localStorage.getItem('usuarioLogado');
      if (usuarioLogado) setEstagiario(JSON.parse(usuarioLogado));
    } else {
      // Valor padrão preventivo para o Mobile, garantindo que a tela não quebre por falta de dados de sess
      setEstagiario({ nome: "Usuário Mobile", cargo: "Estagiário" });
```

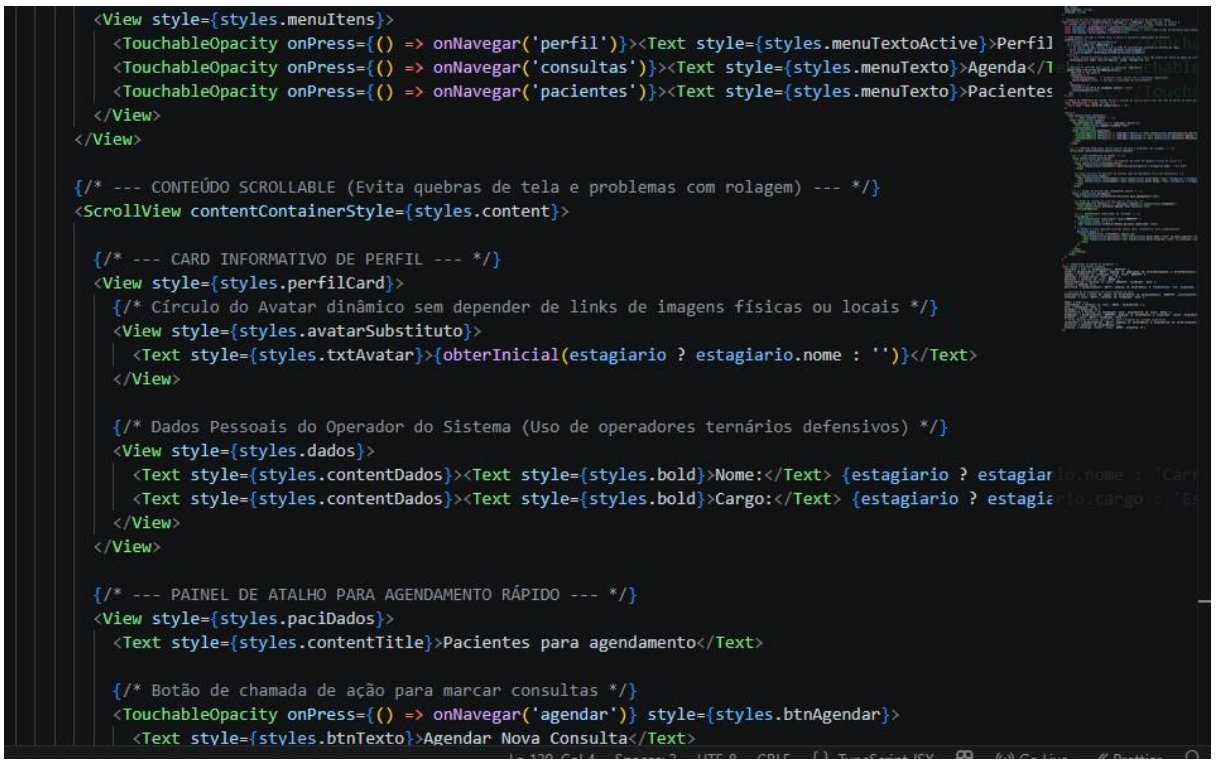


```
    } else {
      // Valor padrão preventivo para o Mobile, garantindo que a tela não quebre por falta de dados de sess
      setEstagiario({ nome: "Usuário Mobile", cargo: "Estagiário" });
    }

    // Requisição HTTP GET para buscar os pacientes cadastrados
    fetch('http://10.13.152.36:5000/pacientes')
      .then(res => res.json())
      .then(data => {
        setPacientes(data);   // Alimenta o mini painel com os pacientes cadastrados
        setCarregando(false); // Desliga o sinalizador de processamento
      })
      .catch(err => {
        console.error("Erro ao alimentar painel:", err);
        setCarregando(false);
      });
  }, []);

  // FUNÇÃO DE FORMATAÇÃO DE AVATAR: Extrai a inicial do usuário para criar uma foto de perfil em vetor de
  const obterInicial = (nome: string) => {
    return nome ? nome.charAt(0).toUpperCase() : 'U';
  };

  return (
    <View style={styles.container}>
      {/* --- MENU SUPERIOR NAVBAR --- */}
      <View style={styles.navbar}>
        <TouchableOpacity onPress={() => onNavegar('perfil')}>
          <Text style={styles.logoNav}>CLINICA</Text>
```



```
        <View style={styles.menuItens}>
          <TouchableOpacity onPress={() => onNavegar('perfil')}><Text style={styles.menuTextoActive}>Perfil
          <TouchableOpacity onPress={() => onNavegar('consultas')}><Text style={styles.menuTexto}>Agenda</T
          <TouchableOpacity onPress={() => onNavegar('pacientes')}><Text style={styles.menuTexto}>Pacientes
        </View>
      </View>

      {/* --- CONTEÚDO SCROLLABLE (Evita quebras de tela e problemas com rolagem) --- */}
      <ScrollView contentContainerStyle={styles.content}>

        {/* --- CARD INFORMATIVO DE PERFIL --- */}
        <View style={styles.perfilCard}>
          {/* Círculo do Avatar dinâmico sem depender de links de imagens físicas ou locais */}
          <View style={styles.avatarSubstituto}>
            <Text style={styles.txtAvatar}>{obterInicial(estagiario ? estagiario.nome : '')}</Text>
          </View>

          {/* Dados Pessoais do Operador do Sistema (Uso de operadores ternários defensivos) */}
          <View style={styles.dados}>
            <Text style={styles.contentDados}><Text style={styles.bold}>Nome:</Text> {estagiario ? estagiar
            <Text style={styles.contentDados}><Text style={styles.bold}>Cargo:</Text> {estagiario ? estagia
          </View>
        </View>

        {/* --- PAINEL DE ATALHO PARA AGENDAMENTO RÁPIDO --- */}
        <View style={styles.paciDados}>
          <Text style={styles.contentTitle}>Pacientes para agendamento</Text>

          {/* Botão de chamada de ação para marcar consultas */}
          <TouchableOpacity onPress={() => onNavegar('agendar')} style={styles.btnAgendar}>
            <Text style={styles.btnTexto}>Agendar Nova Consulta</Text>
```

```
// --- ARQUITETURA DE DESIGN DA INTERFACE ---
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  navbar: { backgroundColor: '#fff', padding: 15, paddingTop: 40, borderBottomWidth: 1, borderBottomColor
  logoNav: { fontSize: 18, fontWeight: 'bold', color: '#007AFF' },
  menuItens: { flexDirection: 'row', gap: 15 },
  menuTexto: { fontSize: 14, color: '#666' },
  menuTextoActive: { fontSize: 14, color: '#007AFF', fontWeight: 'bold' },
  content: { padding: 15 },
  perfilCard: { backgroundColor: '#fff', padding: 15, borderRadius: 8, flexDirection: 'row', alignItems:

  // Estilização do componente de Avatar baseado em texto
  avatarSubstituto: { width: 50, height: 50, borderRadius: 25, backgroundColor: '#007AFF', justifyContent
  txtAvatar: { color: '#fff', fontSize: 20, fontWeight: 'bold' },

  dados: { flex: 1 },
  contentDados: { fontSize: 16, color: '#333', marginBottom: 4 },
  bold: { fontWeight: 'bold' },
  paciDados: { marginTop: 10 },
  contentTitle: { fontSize: 18, fontWeight: 'bold', marginBottom: 15, color: '#333' },
  btnAgendar: { backgroundColor: '#007AFF', padding: 12, borderRadius: 6, alignItems: 'center', marginBot
  btnTexto: { color: '#fff', fontWeight: 'bold' },
  // Borda esquerda destacada para dar um aspecto elegante de listagem corporativa
  contentPaci: { backgroundColor: '#fff', padding: 12, borderRadius: 6, marginBottom: 10, borderLeftWidth
  paciTexto: { fontSize: 14, marginBottom: 2 },
  txtVazio: { textAlign: 'center', color: '#999', marginTop: 10 }
});
```

Back-End/API:

```
Site-Principal > backend > app.py > ...
from flask import Flask, request, jsonify
from flask_cors import CORS
import mysql.connector
import datetime

# Inicialização do microframework Flask
app = Flask(__name__)

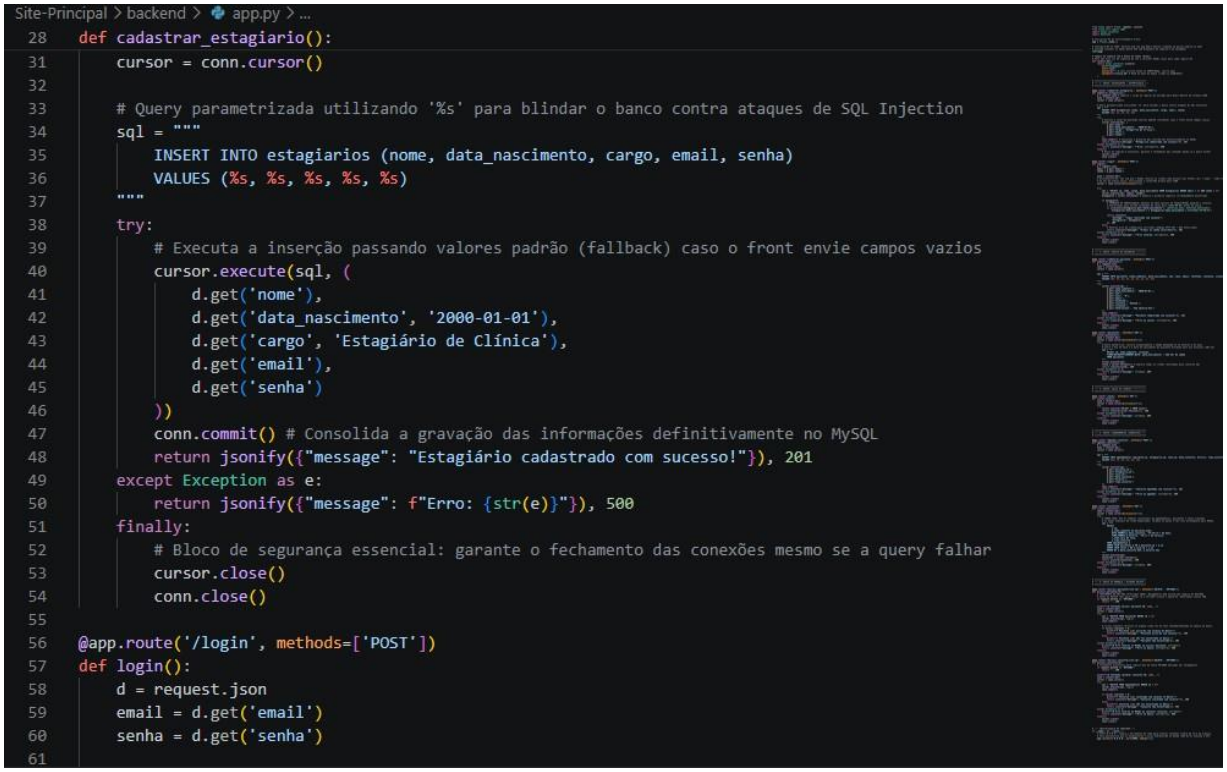
# Configuração do CORS: Permite que seu app React Native (rodando em portas mobile ou web)
# consiga consumir os dados desta API sem bloqueios de segurança do navegador
CORS(app)

# FUNÇÃO DE CONEXÃO COM O BANCO DE DADOS (MySQL)
# Abre uma nova via de comunicação com o servidor MySQL local para cada requisição
def connect_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="", # Caso utilize senha no XAMPP/Wamp, insira aqui
        database="clinica_db" # Nome da base de dados criada no phpMyAdmin
    )

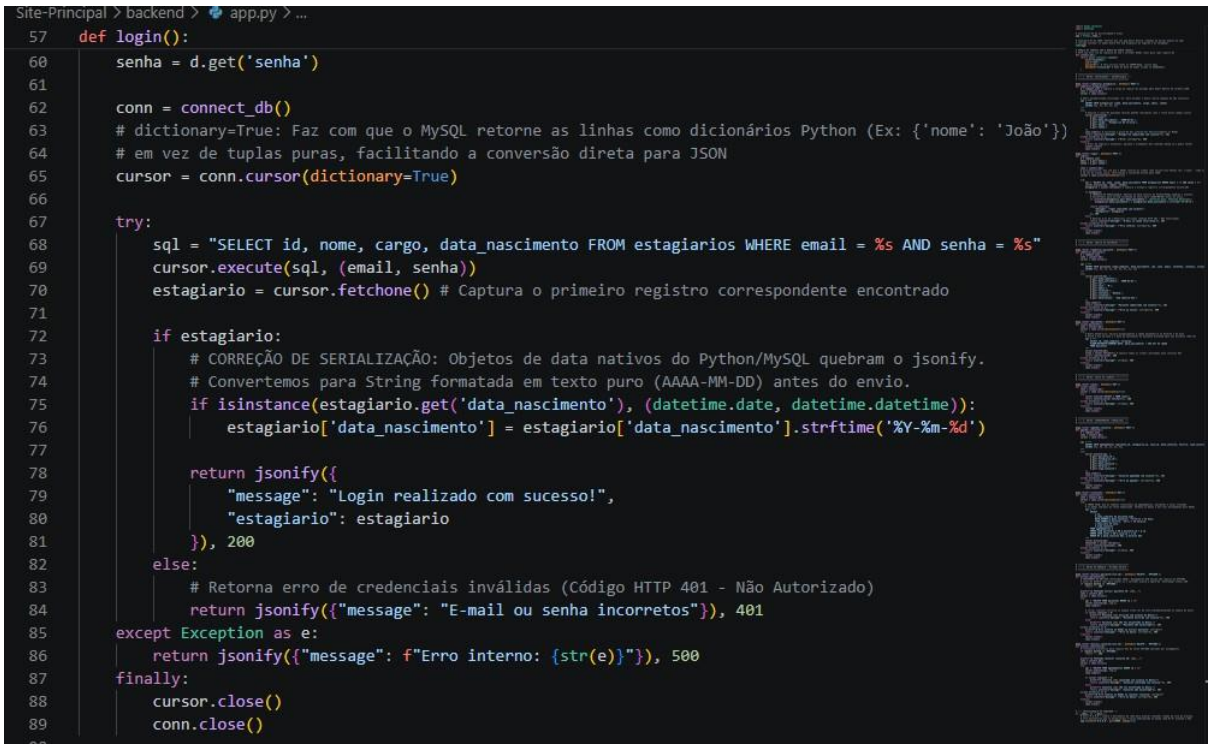
# ========================================
# --- 1. ROTAS: ESTAGIÁRIOS / AUTENTICAÇÃO ---
# ========================================

@app.route('/cadastrar_estagiario', methods=['POST'])
def cadastrar_estagiario():
    d = request.json # Captura o corpo da requisição enviado pelo React Native em formato JSON
    conn = connect_db()
    cursor = conn.cursor()

```



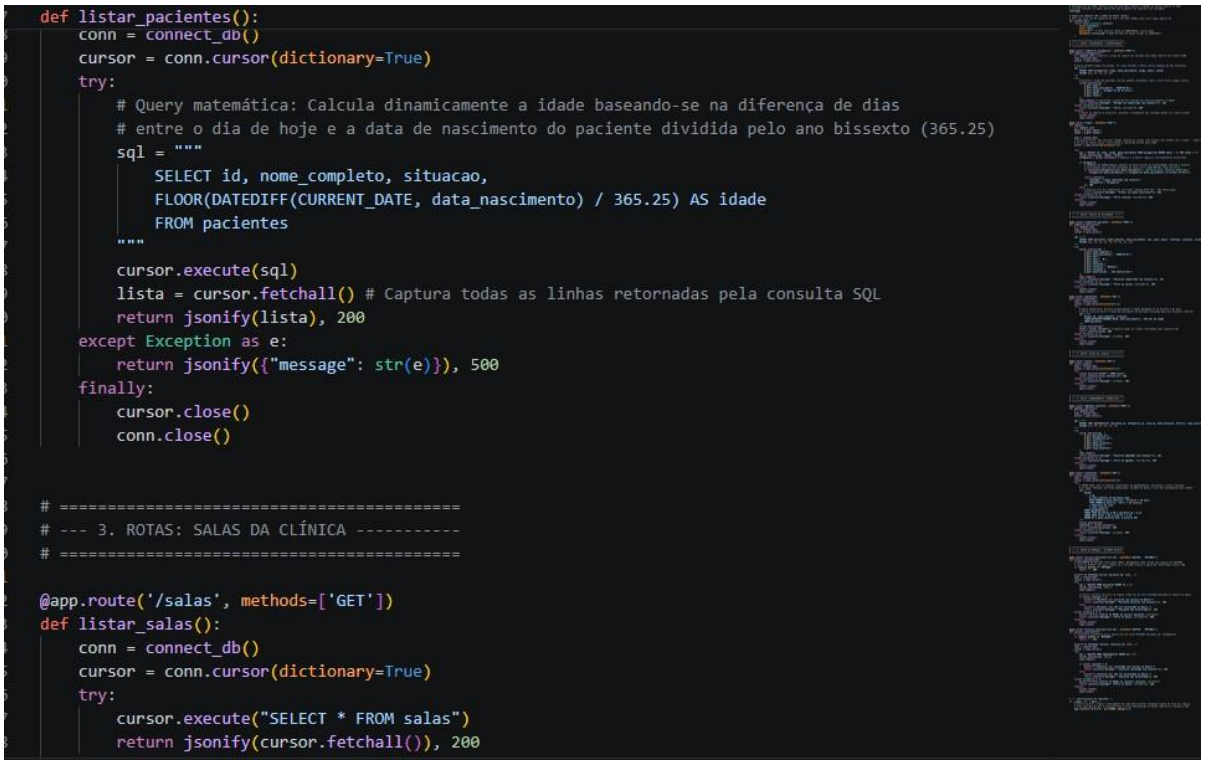
```
Site-Principal > backend > app.py > ...
28  def cadastrar_estagiario():
31      cursor = conn.cursor()
32
33      # Query parametrizada utilizando '%s' para blindar o banco contra ataques de SQL Injection
34      sql = """
35          INSERT INTO estagiarios (nome, data_nascimento, cargo, email, senha)
36          VALUES (%s, %s, %s, %s, %s)
37      """
38      try:
39          # Executa a inserção passando valores padrão (fallback) caso o front envie campos vazios
40          cursor.execute(sql, (
41              d.get('nome'),
42              d.get('data_nascimento', '2000-01-01'),
43              d.get('cargo', 'Estagiário de Clínica'),
44              d.get('email'),
45              d.get('senha')
46          ))
47          conn.commit() # Consolida a gravação das informações definitivamente no MySQL
48          return jsonify({"message": "Estagiário cadastrado com sucesso!"}), 201
49      except Exception as e:
50          return jsonify({"message": f"Erro: {str(e)}"}), 500
51      finally:
52          # Bloco de segurança essencial: garante o fechamento das conexões mesmo se a query falhar
53          cursor.close()
54          conn.close()
55
56  @app.route('/login', methods=['POST'])
57  def login():
58      d = request.json
59      email = d.get('email')
60      senha = d.get('senha')
61
```



```
Site-Principal > backend > app.py > ...
57  def login():
60      senha = d.get('senha')
61
62      conn = connect_db()
63      # dictionary=True: Faz com que o MySQL retorne as linhas como dicionários Python (Ex: {'nome': 'João'})
64      # em vez de tuplas puras, facilitando a conversão direta para JSON
65      cursor = conn.cursor(dictionary=True)
66
67      try:
68          sql = "SELECT id, nome, cargo, data_nascimento FROM estagiarios WHERE email = %s AND senha = %s"
69          cursor.execute(sql, (email, senha))
70          estagiario = cursor.fetchone() # Captura o primeiro registro correspondente encontrado
71
72          if estagiario:
73              # CORREÇÃO DE SERIALIZAÇÃO: Objetos de data nativos do Python/MySQL quebram o jsonify.
74              # Convertemos para String formatada em texto puro (AAAA-MM-DD) antes do envio.
75              if isinstance(estagiario.get('data_nascimento'), (datetime.date, datetime.datetime)):
76                  estagiario['data_nascimento'] = estagiario['data_nascimento'].strftime('%Y-%m-%d')
77
78              return jsonify({
79                  "message": "Login realizado com sucesso!",
80                  "estagiario": estagiario
81              }), 200
82          else:
83              # Retorna erro de credenciais inválidas (Código HTTP 401 - Não Autorizado)
84              return jsonify({"message": "E-mail ou senha incorretos"}), 401
85      except Exception as e:
86          return jsonify({"message": f"Erro interno: {str(e)}"}), 500
87      finally:
88          cursor.close()
89          conn.close()
```



```
Site-Principal > backend > app.py > ...
93   # --- 2. ROTAS: GESTÃO DE PACIENTES --------
94   # ========================================
95
96   @app.route('/cadastrar_paciente', methods=['POST'])
97   def cadastrar_paciente():
98       d = request.json
99       conn = connect_db()
100      cursor = conn.cursor()
101
102      sql = """
103          INSERT INTO pacientes (nome_completo, data_nascimento, cpf, sexo, email, telefone, convenio, situac
104          VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
105      """
106      try:
107          cursor.execute(sql, (
108              d.get('nome_completo'),
109              d.get('data_nascimento', '2000-01-01'),
110              d.get('cpf'),
111              d.get('sexo', 'M'),
112              d.get('email'),
113              d.get('telefone'),
114              d.get('convenio', 'Nenhum'),
115              d.get('situacao'),
116              d.get('observacoes', 'Sem observações')
117          ))
118          conn.commit()
119          return jsonify({"message": "Paciente cadastrado com sucesso!"}), 201
120      except Exception as e:
121          return jsonify({"message": f"Erro ao salvar: {str(e)}"}), 500
122      finally:
123          cursor.close()
124          conn.close()
```



```
def listar_pacientes():
    conn = connect_db()
    cursor = conn.cursor(dictionary=True)
    try:
        # Query matemática: Calcula dinamicamente a idade baseando-se na diferença de dias
        # entre o dia de hoje e a data de nascimento do paciente dividida pelo ano bissexto (365.25)
        sql = """
            SELECT id, nome_completo, situacao,
            FLOOR(DATEDIFF(CURRENT_DATE, data_nascimento) / 365.25) AS idade
            FROM pacientes
        """
        cursor.execute(sql)
        lista = cursor.fetchall() # Captura todas as linhas retornadas pela consulta SQL
        return jsonify(lista), 200
    except Exception as e:
        return jsonify({"message": str(e)}), 500
    finally:
        cursor.close()
        conn.close()


# ========================================
# --- 3. ROTAS: SALAS DA CLÍNICA ----------
# ========================================

@app.route('/salas', methods=['GET'])
def listar_salas():
    conn = connect_db()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute("SELECT * FROM salas")
        return jsonify(cursor.fetchall()), 200
```

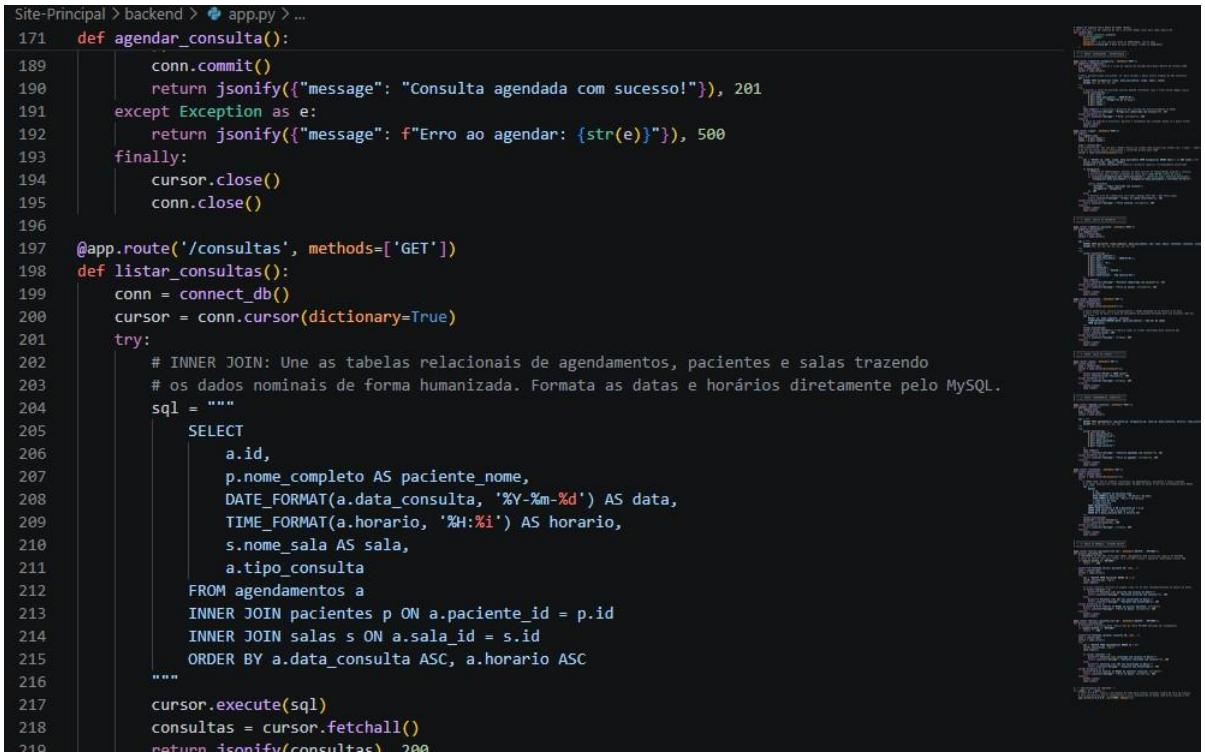


```
def listar_salas():
        cursor.execute("SELECT * FROM salas")
        return jsonify(cursor.fetchall()), 200
    except Exception as e:
        return jsonify({"message": str(e)}), 500
    finally:
        cursor.close()
        conn.close()


# ========================================
# --- 4. ROTAS: AGENDAMENTOS (CONSULTAS) ---
# ========================================

@app.route('/agendar_consulta', methods=['POST'])
def agendar_consulta():
    d = request.json
    conn = connect_db()
    cursor = conn.cursor()

    sql = """
        INSERT INTO agendamentos (paciente_id, estagiario_id, sala_id, data_consulta, horario, tipo_consult
        VALUES (%s, %s, %s, %s, %s, %s)
    """
    try:
        cursor.execute(sql, (
            d.get('paciente_id'),
            d.get('estagiario_id'),
            d.get('sala_id'),
            d.get('data_consulta'),
            d.get('horario'),
            d.get('tipo_consulta')
```



```
Site-Principal > backend > app.py > ...
171   def agendar_consulta():
189           conn.commit()
190           return jsonify({"message": "Consulta agendada com sucesso!"}), 201
191       except Exception as e:
192           return jsonify({"message": f"Erro ao agendar: {str(e)}"}), 500
193       finally:
194           cursor.close()
195           conn.close()
196
197   @app.route('/consultas', methods=['GET'])
198   def listar_consultas():
199       conn = connect_db()
200       cursor = conn.cursor(dictionary=True)
201       try:
202           # INNER JOIN: Une as tabelas relacionais de agendamentos, pacientes e salas trazendo
203           # os dados nominais de forma humanizada. Formata as datas e horários diretamente pelo MySQL.
204           sql = """
205               SELECT
206                   a.id,
207                   p.nome_completo AS paciente_nome,
208                   DATE_FORMAT(a.data_consulta, '%Y-%m-%d') AS data,
209                   TIME_FORMAT(a.horario, '%H:%i') AS horario,
210                   s.nome_sala AS sala,
211                   a.tipo_consulta
212               FROM agendamentos a
213               INNER JOIN pacientes p ON a.paciente_id = p.id
214               INNER JOIN salas s ON a.sala_id = s.id
215               ORDER BY a.data_consulta ASC, a.horario ASC
216           """
217           cursor.execute(sql)
218           consultas = cursor.fetchall()
219           return jsonify(consultas), 200
```

```
def listar_consultas():
    except Exception as e:
        return jsonify({"message": str(e)}), 500
    finally:
        cursor.close()
        conn.close()


# ========================================
# --- 5. ROTAS DE REMOÇÃO / MÉTODOS DELETE -
# ========================================

@app.route('/excluir_paciente/<int:id>', methods=['DELETE', 'OPTIONS'])
def excluir_paciente(id):
    # TRATAMENTO DE PRÉ-VOO (Preflight CORS): Navegadores Web enviam uma requisição OPTIONS
    # antes do DELETE real para checar se o servidor aceita a operação. Retornamos status 200.
    if request.method == 'OPTIONS':
        return '', 200

    print(f"🗑️ Tentando excluir paciente ID: {id}...")
    conn = connect_db()
    cursor = conn.cursor()
    try:
        sql = "DELETE FROM pacientes WHERE id = %s"
        cursor.execute(sql, (id,))
        conn.commit()

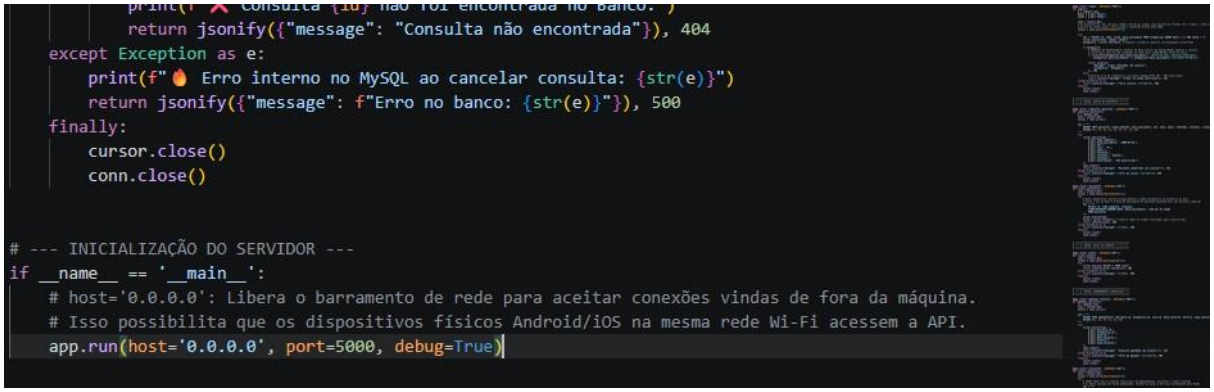
        # cursor.rowcount verifica se alguma linha foi de fato afetada/deletada na tabela do banco
        if cursor.rowcount > 0:
            print(f"✔ Paciente {id} excluído com sucesso do Banco!")
            return jsonify({"message": "Paciente excluído com sucesso!"}), 200
        else:
```

```
def excluir_paciente(id):
            return jsonify({"message": "Paciente não encontrado"}), 404
    except Exception as e:
        print(f"🔥 Erro interno no MySQL ao excluir paciente: {str(e)}")
        return jsonify({"message": f"Erro no banco: {str(e)}"}), 500
    finally:
        cursor.close()
        conn.close()

@app.route('/excluir_consulta/<int:id>', methods=['DELETE', 'OPTIONS'])
def excluir_consulta(id):
    # Tratamento preventivo para requisições de teste OPTIONS emitidas por navegadores
    if request.method == 'OPTIONS':
        return '', 200

    print(f"🗑️ Tentando cancelar consulta ID: {id}...")
    conn = connect_db()
    cursor = conn.cursor()
    try:
        sql = "DELETE FROM agendamentos WHERE id = %s"
        cursor.execute(sql, (id,))
        conn.commit()

        if cursor.rowcount > 0:
            print(f"✔ Consulta {id} cancelada com sucesso no Banco!")
            return jsonify({"message": "Consulta cancelada com sucesso!"}), 200
        else:
            print(f"❌ Consulta {id} não foi encontrada no Banco.")
            return jsonify({"message": "Consulta não encontrada"}), 404
    except Exception as e:
        print(f"🔥 Erro interno no MySQL ao cancelar consulta: {str(e)}")
        return jsonify({"message": f"Erro no banco: {str(e)}"}), 500
```



```
            print(f"❌ Consulta {id} não foi encontrada no banco.")
            return jsonify({"message": "Consulta não encontrada"}), 404
    except Exception as e:
        print(f"🔥 Erro interno no MySQL ao cancelar consulta: {str(e)}")
        return jsonify({"message": f"Erro no banco: {str(e)}"}), 500
    finally:
        cursor.close()
        conn.close()


# --- INICIALIZAÇÃO DO SERVIDOR ---
if __name__ == '__main__':
    # host='0.0.0.0': Libera o barramento de rede para aceitar conexões vindas de fora da máquina.
    # Isso possibilita que os dispositivos físicos Android/iOS na mesma rede Wi-Fi acessem a API.
    app.run(host='0.0.0.0', port=5000, debug=True)
```

1. Aceite da Empresa

Termo de aceite: conteúdo administrativo omitido nesta versão pública para preservar dados pessoais e informações de assinatura.

O texto administrativo de revisão e aceite foi omitido nesta versão pública. A versão disponibilizada tem finalidade de portfólio acadêmico e não substitui documentos oficiais da instituição.

Informações administrativas sobre doação, ajustes, manutenção e garantia foram omitidas nesta versão pública. Consulte a instituição responsável para informações oficiais.

Declaração de aceite e identificação dos signatários omitidas nesta versão pública.

Data:

Assinaturas:

[Nome do representante omitido na versão pública]		[Identificação do signatário omitida]

Anexo administrativo editado para publicação pública, com identificação dos signatários e dados de aceite omitidos.